

BURSA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ



UniversalERP Sistemi:
Çok Kiracılı Kurumsal Kaynak Planlama
Uygulamasının Tasarımı ve İmplementasyonu

LİSANS BİTİRME ÇALIŞMASI
Öğrenci Talha YÜKSEL

Bilgisayar Mühendisliği Bölümü

HAZİRAN 2026

BURSA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ



UniversalERP Sistemi:
Çok Kiracılı Kurumsal Kaynak Planlama
Uygulamasının Tasarımı ve İmplementasyonu

LİSANS BİTİRME ÇALIŞMASI
Öğrenci Talha YÜKSEL

Bilgisayar Mühendisliği Bölümü

HAZİRAN 2026

BTÜ, Mühendislik ve Doğa Bilimleri Fakültesi Bilgisayar Mühendisliği Bölümü'nün 21360859038 numaralı öğrencisi Talha YÜKSEL, ilgili yönetmeliklerin belirlediği gerekli tüm şartları yerine getirdikten sonra hazırladığı "UniversalERP Sistemi: Çok Kiracılı Kurumsal Kaynak Planlama Uygulamasının Tasarımı ve İmplementasyonu" başlıklı bitirme çalışmasını aşağıda imzaları olan jüri önünde başarı ile sunmuştur.

Danışmanı : **Prof. Dr. Adı SOYADI** Doç. Dr. Ergün GÜMÜŞ
Bursa Teknik Üniversitesi

Jüri Üyeleri : **Dr. Öğr. Üyesi Adı SOYADI**
Bursa Teknik Üniversitesi

Arş. Gör. Adı SOYADI
Bursa Teknik Üniversitesi

Savunma Tarihi : 28 Haziran 2026

BM Bölüm Başkanı : Prof. Dr. Adı Soyadı
Prof. Dr. Haydar ÖZKAN
Bursa Teknik Üniversitesi/...../.....

İNTİHAL BEYANI

Bu bitirme çalışmasında sunulan tüm bilgi, belge, görsel materyal ve sonuçların akademik ve etik kurallara uygun olarak elde edildiğini; kullanılan tüm kaynaklara metin içinde ve kaynakçada eksiksiz atıfta bulunulduğunu; başkalarına ait özgün ifadelerin alıntı kuralları çerçevesinde tırnak içinde gösterildiğini; bu çalışmanın herhangi bir bölümünün bu üniversitede veya başka bir üniversitede başka bir bitirme çalışması, tez veya proje olarak sunulmadığını; ve bu çalışmanın tümüyle tarafımdan, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü'nde hazırlanmış özgün bir eser olduğunu beyan ederim.

Aksi bir durumun tespiti halinde doğabilecek tüm hukuki ve akademik sonuçları kabul ettiğimi taahhüt ederim.

Öğrencinin Adı Soyadı: Talha YÜKSEL

İmzası :

Açıklamalı [BB1]:

Mavi kalemle imzalanacaktır

(iii) sayılır ancak sayfa numarası yazılmaz

ÖNSÖZ

Bu bitirme çalışması, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü'ndeki dört yıllık lisans öğrenimimin teorik birikimini somut ve eksiksiz bir yazılım ürününe dönüştürme amacıyla hazırlanmıştır. Çalışma kapsamında geliştirilen UniversalERP sistemi, modern yazılım mimarisi prensiplerini, bulut tabanlı dağıtım tekniklerini ve katmanlı bir geliştirme yaklaşımını tek bir bütünlük üründe buluşturmaktadır.

Kurumsal Kaynak Planlama (ERP) sistemleri, işletmelerin en kritik dijital omurgalarından birini oluşturur. Bu çalışmada, büyük ölçekli ticari ERP çözümlerinin sahip olduğu temel yeteneklerin önemli bir bölümünü; tek bir uygulama örneği üzerinden birden fazla firmaya hizmet verebilen çok kiracılı (multi-tenant) bir mimari ile, açık kaynaklı teknolojiler kullanarak ve ölçeklenebilir bir biçimde gerçekleştirmek hedeflenmiştir.

Proje sürecinde Clean Architecture, SOLID prensipleri, Repository ve Unit of Work desenleri, JWT tabanlı kimlik doğrulama, rol tabanlı erişim denetimi ve kapsamlı bir denetim günlüğü (audit log) mekanizması gibi pek çok ileri düzey konuyu uygulamalı olarak deneyimleme fırsatı buldum. Bu deneyim, yazılım mühendisliğini bir derleme bilgi yığını olmaktan çıkarıp, gerçek mühendislik kararlarının verildiği bir disipline dönüştürdü.

Çalışmam boyunca değerli yönlendirmeleri, yapıcı eleştirileri ve akademik desteğiyle bana yol gösteren danışman hocama içtenlikle teşekkür ederim. Ayrıca lisans öğrenim süresince bilgi ve deneyimlerini benimle paylaşan tüm bölüm hocalarıma, bu uzun ve zorlu süreçte maddi ve manevi desteğini hiçbir zaman esirgemeyen aileme şükranlarımı sunarım.

Haziran 2026

Talha Yüksel

Açıklamalı [BTÜ-FBE2]:

Tarih ve yazar isminin aynı hizada olması gerekir.

İÇİNDEKİLER

İNTİHAL BEYANI	i
ÖNSÖZ	ii
İÇİNDEKİLER.....	iii
ŞEKİL LİSTESİ	v
ÇİZELGE LİSTESİ	vi
KISALTMALAR	vii
ÖZET	viii
ABSTRACT	ix
1. GİRİŞ.....	1
1.1. Problemin Tanımı ve Çalışmanın Önemi	1
1.2. Çalışmanın Amacı ve Hedefleri	2
1.3. Literatür ve Mevcut Çözümlerin Değerlendirilmesi	3
1.4. Çalışmanın Kapsamı ve Sınırlılıkları	4
1.5. Tezin Organizasyonu	5
2. KULLANILAN TEKNOLOJİLER VE YÖNTEMLER	6
2.1. Backend Teknolojileri	6
2.2. Frontend Teknolojileri.....	8
2.3. Veritabanı Yönetim Sistemi	9
2.4. Yardımcı Kütüphaneler ve Araçlar.....	10
2.5. Geliştirme Metodolojisi	11
3. SİSTEM MİMARİSİ VE TASARIMI.....	12
3.1. Clean Architecture Yaklaşımı	12
3.2. Katmanlı Mimari Yapısı.....	14
3.3. Tasarım Desenleri	16
3.4. SOLID Prensiplerinin Uygulanması.....	18
3.5. Çok Kiracılı (Multi-Tenant) Mimari	19
4. VERİTABANI TASARIMI	22
4.1. Varlık-İlişki Modeli	22
4.2. Tablo Yapıları ve İlişkiler.....	24
4.3. Veri İzolasyonu ve Global Query Filters.....	26
4.4. İndeksleme ve Performans Optimizasyonu	27
5. UYGULAMANIN GERÇEKLEŞTİRİLMESİ	29
5.1. Kimlik Doğrulama ve Oturum Yönetimi.....	29
5.2. Kontrol Paneli (Dashboard) Modülü	32
5.3. Ürün Yönetimi Modülü	35

5.4. Müşteri Yönetimi Modülü	37
5.5. Satış Yönetimi Modülü.....	39
5.6. Firma Ayarları ve Kiracı Yönetimi	41
5.7. İşlem Geçmişi (Audit Log) Modülü	43
6. GÜVENLİK VE YETKİLENDİRME	45
6.1. JWT Tabanlı Kimlik Doğrulama	45
6.2. Rol Tabanlı Erişim Denetimi (RBAC)	46
6.3. Veri Güvenliği Önlemleri	47
7. TEST VE PERFORMANS DEĞERLENDİRMESİ	49
7.1. Test Stratejisi	49
7.2. Performans Analizi	50
8. SONUÇLAR VE ÖNERİLER.....	52
8.1. Elde Edilen Sonuçlar	52
8.2. Karşılaşılan Zorluklar	53
8.3. Gelecek Çalışmalar.....	54
KAYNAKLAR.....	55
ÖZGEÇMİŞ	57

ŞEKİL LİSTESİ

Şekil 1.1. Türkiye'de KOBİ'lerin dijitalleşme oranları	3
Şekil 3.1. Clean Architecture katman diyagramı	13
Şekil 3.2. UniversalERP katmanlı mimari yapısı	15
Şekil 3.3. Çok kiracılı veri izolasyonu akış şeması	20
Şekil 4.1. UniversalERP varlık-ilişki (ER) diyagramı	23
Şekil 5.1. Sisteme giriş (login) ekranı	30
Şekil 5.2. Kontrol paneli - aydınlık tema	33
Şekil 5.3. Kontrol paneli - karanlık tema	34
Şekil 5.4. Ürün yönetimi ekranı ve kritik stok uyarıları	36
Şekil 5.5. Müşteri yönetimi ekranı	38
Şekil 5.6. Satış yönetimi ekranı	40
Şekil 5.7. Firma ayarları ve kiracı bilgileri ekranı	42
Şekil 5.8. İşlem geçmişi (audit log) ekranı	44

ÇİZELGE LİSTESİ

Çizelge 2.1. Kullanılan backend teknolojileri ve sürümleri	7
Çizelge 2.2. Kullanılan frontend teknolojileri ve sürümleri	8
Çizelge 3.1. Sistemdeki kullanıcı rolleri ve yetki düzeyleri	21
Çizelge 4.1. Temel veritabanı tabloları ve açıklamaları	25
Çizelge 5.1. REST API uç noktalarından örnekler	31
Çizelge 7.1. Yük testi performans sonuçları	51

KISALTMALAR

API Arayüzü)	: Application Programming Interface (Uygulama Programlama
CRUD	: Create, Read, Update, Delete (Oluştur, Oku, Güncelle, Sil)
DTO	: Data Transfer Object (Veri Transfer Nesnesi)
EF Core	: Entity Framework Core
ERP	: Enterprise Resource Planning (Kurumsal Kaynak Planlama)
HTTP	: HyperText Transfer Protocol
JWT	: JSON Web Token
KOBİ	: Küçük ve Orta Büyüklükteki İşletme
ORM	: Object-Relational Mapping (Nesne-İlişkisel Eşleme)
RBAC	: Role-Based Access Control (Rol Tabanlı Erişim Denetimi)
REST	: Representational State Transfer
SPA	: Single Page Application (Tek Sayfa Uygulaması)
SQL	: Structured Query Language
UI/UX	: User Interface / User Experience
XSS	: Cross-Site Scripting

ÖZET

UNIVERSALERP: ÇOK KIRACILI BİR KURUMSAL KAYNAK PLANLAMA (ERP) SİSTEMİNİN BULUT TABANLI MİMARİYLE TASARIMI VE GERÇEKLEŞTİRİLMESİ

Kurumsal Kaynak Planlama (ERP) sistemleri, işletmelerin insan kaynakları, satış, stok, müşteri ilişkileri ve muhasebe gibi temel iş süreçlerini tek bir bütünsel platformda yöneterek operasyonel verimliliği artıran kritik yazılım çözümleridir. Ancak piyasadaki yerleşik ERP çözümleri yüksek lisanslama maliyetleri, uzun ve karmaşık kurulum süreçleri ve sınırlı esneklikleri nedeniyle özellikle küçük ve orta büyüklükteki işletmeler (KOBİ) için çoğu zaman erişilebilir olmaktan uzaktır.

Bu bitirme çalışması kapsamında, bu boşluğu doldurmayı hedefleyen UniversalERP adında, çok kiracılı (multi-tenant) mimariye sahip, bulut tabanlı ve tam işlevsel bir ERP sistemi tasarlanmış ve gerçekleştirilmiştir. Sistem, sunucu tarafında ASP.NET Core 8 ve Entity Framework Core, istemci tarafında ise Vue.js 3 ve Pinia teknolojileri kullanılarak geliştirilmiştir. Veri katmanında Microsoft SQL Server tercih edilmiştir.

Yazılım mimarisi, sürdürülebilirliği ve test edilebilirliği ön planda tutan Clean Architecture yaklaşımı üzerine inşa edilmiştir. Repository ve Unit of Work tasarım desenleri ile veri erişim katmanı soyutlanmış, SOLID prensipleri ile bağımlılıklar yönetilmiştir. Çok kiracılı yapı, Entity Framework Core'un Global Query Filters özelliği aracılığıyla veritabanı düzeyinde otomatik kiracı izolasyonu sağlanarak gerçekleştirilmiştir. Böylece tek bir uygulama ve veritabanı örneği üzerinden birden çok firmaya, verileri tamamen birbirinden ayrılmış biçimde hizmet verilebilmektedir.

Güvenlik katmanında JSON Web Token (JWT) tabanlı kimlik doğrulama ve altı farklı yetki düzeyine sahip rol tabanlı erişim denetimi (RBAC) uygulanmıştır. Tüm kritik veri değişiklikleri, kim tarafından ve ne zaman yapıldığı bilgisiyle birlikte denetim günlüğüne (audit log) kaydedilmektedir. Sistem; ürün yönetimi, müşteri yönetimi, satış işlemleri, kritik stok takibi, gelir analizi ve raporlama gibi kapsamlı modüller içermektedir.

Geliştirilen sistem, modern yazılım mühendisliği prensiplerinin gerçek bir uygulamada nasıl bir araya getirilebileceğini somut biçimde ortaya koymakta; KOBİ'ler için düşük maliyetli, ölçeklenebilir ve genişletilebilir bir ERP alternatifi sunmaktadır.

Anahtar Kelimeler: Kurumsal Kaynak Planlama, Çok Kiracılı Mimari, Clean Architecture, ASP.NET Core, Vue.js, Bulut Bilişim

ABSTRACT

UNIVERSALERP: DESIGN AND IMPLEMENTATION OF A MULTI-TENANT ENTERPRISE RESOURCE PLANNING (ERP) SYSTEM WITH A CLOUD-BASED ARCHITECTURE

Enterprise Resource Planning (ERP) systems are critical software solutions that increase operational efficiency by managing core business processes such as human resources, sales, inventory, customer relations, and accounting on a single integrated platform. However, established ERP solutions on the market are often inaccessible, particularly for small and medium-sized enterprises (SMEs), due to high licensing costs, lengthy and complex deployment processes, and limited flexibility.

Within the scope of this graduation project, a fully functional, cloud-based ERP system named UniversalERP with a multi-tenant architecture has been designed and implemented to address this gap. The system was developed using ASP.NET Core 8 and Entity Framework Core on the server side, and Vue.js 3 with Pinia on the client side. Microsoft SQL Server was chosen for the data layer.

The software architecture is built upon the Clean Architecture approach, which prioritizes maintainability and testability. The data access layer is abstracted through the Repository and Unit of Work design patterns, and dependencies are managed according to SOLID principles. The multi-tenant structure is realized by providing automatic tenant isolation at the database level via Entity Framework Core's Global Query Filters feature. Thus, multiple companies can be served through a single application and database instance with their data completely segregated.

In the security layer, JSON Web Token (JWT) based authentication and role-based access control (RBAC) with six different permission levels are applied. All critical data changes are recorded in an audit log along with information on who made them and when. The system includes comprehensive modules such as product management, customer management, sales transactions, critical stock tracking, revenue analysis, and reporting.

The developed system concretely demonstrates how modern software engineering principles can be combined in a real-world application, offering a low-cost, scalable, and extensible ERP alternative for SMEs.

Keywords: Enterprise Resource Planning, Multi-Tenant Architecture, Clean Architecture, ASP.NET Core, Vue.js, Cloud Computing

1. GİRİŞ

Dijital dönüşümün hız kazandığı günümüz iş dünyasında, işletmelerin rekabet gücünü koruyabilmesi büyük ölçüde iş süreçlerini ne kadar verimli yönetebildiklerine bağlıdır. Geçmişte birbirinden bağımsız tablolar, kağıt belgeler ve dağınık yazılımlarla yürütülen muhasebe, stok, satış ve müşteri yönetimi gibi faaliyetler, artık bütünsel yazılım platformları üzerinden tek bir merkezden yönetilmektedir. Bu bütünsel yönetimi sağlayan yazılım sınıfının adı Kurumsal Kaynak Planlama (Enterprise Resource Planning, ERP) sistemleridir.

Bir ERP sistemi, bir kuruluşun farklı departmanlarına ait verileri ortak bir veri modeli ve merkezi bir veritabanı etrafında toplar. Böylece satış departmanının girdiği bir sipariş, anında stok seviyesini güncelleyebilir, muhasebe kayıtlarını tetikleyebilir ve yönetim için anlamlı raporlara dönüşebilir. Bu bütünsel yapı, veri tekrarını ortadan kaldırır, hata oranını düşürür ve karar alma süreçlerini hızlandırır.

1.1. Problemin Tanımı ve Çalışmanın Önemi

Piyasada SAP, Oracle ERP Cloud ve Microsoft Dynamics 365 gibi son derece güçlü ve kapsamlı ticari ERP çözümleri bulunmaktadır. Ancak bu çözümler, kurumsal ölçekteki büyük işletmeler düşünülerek tasarlanmıştır. Yüksek lisanslama ve danışmanlık maliyetleri, aylar süren kurulum ve uyarlama süreçleri ve kullanımı için gereken uzman personel ihtiyacı, bu sistemleri küçük ve orta büyüklükteki işletmeler (KOBİ) için çoğunlukla ulaşılabilir kılmamaktadır.

Oysa ekonominin önemli bir bölümünü oluşturan KOBİ'lerin de stok takibi, satış yönetimi, müşteri ilişkileri ve gelir analizi gibi temel ERP işlevlerine ihtiyacı vardır. Bu işletmeler çoğunlukla ya bu ihtiyaçlarını dağınık ve birbirine entegre olmayan araçlarla (örneğin elektronik tablolar) karşılamaya çalışmakta ya da süreçlerini hiç dijitalleştirememektedir. Bu durum, veri kaybına, hatalı kararlara ve verimsizliğe yol açmaktadır.

Bu çalışmanın temel motivasyonu, bu boşluğu doldurmaktır. Modern, açık kaynaklı ve bulut tabanlı teknolojilerin doğru bir mimari ile bir araya getirilmesiyle; düşük maliyetli, kolay dağıtılabilir ve ölçeklenebilir bir ERP çözümünün gerçekleştirilebileceği gösterilmektedir. Çalışmanın en kritik yönü, tek bir uygulama örneğinin birden çok firmaya güvenli biçimde hizmet verebilmesini sağlayan çok kiracılı (multi-tenant) mimaridir. Bu mimari sayesinde, hizmet sağlayıcı tek bir altyapıyı yöneterek maliyetleri minimumda tutabilir ve bu tasarrufu son kullanıcıya yansıtabilir.

1.2. Çalışmanın Amacı ve Hedefleri

Bu bitirme çalışmasının temel amacı, üretim ortamında çalışmaya elverişli, çok kiracılı, güvenli ve ölçeklenebilir bir ERP sisteminin uçtan uca tasarlanması ve gerçekleştirilmesidir.

Bu temel amaç doğrultusunda belirlenen somut hedefler şunlardır:

1. Clean Architecture prensiplerine dayalı, katmanları net biçimde ayrılmış, sürdürülebilir ve test edilebilir bir yazılım mimarisi kurmak.
2. Tek bir uygulama ve veritabanı örneği üzerinden birden fazla firmaya, verileri tam izolasyonla hizmet veren çok kiracılı bir altyapı geliştirmek.
3. JWT tabanlı kimlik doğrulama ve rol tabanlı erişim denetimi ile kurumsal düzeyde bir güvenlik katmanı oluşturmak.
4. Ürün, müşteri, satış, stok ve raporlama gibi temel ERP modüllerini eksiksiz biçimde gerçekleştirmek.
5. Tüm kritik işlemleri izlenebilir kılan kapsamlı bir denetim günlüğü (audit log) mekanizması tasarlamak.
6. Kullanıcı deneyimini ön planda tutan, duyarlı (responsive) ve tema desteği sunan modern bir kullanıcı arayüzü geliştirmek.

1.3. Literatür ve Mevcut Çözümlerin Değerlendirilmesi

ERP kavramı, 1990'lı yıllarda üretim kaynak planlama (MRP II) sistemlerinin tüm işletme süreçlerini kapsayacak biçimde genişlemesiyle ortaya çıkmıştır. İlk nesil ERP sistemleri, işletmelerin kendi sunucularına kurdukları, yüksek maliyetli ve esnek olmayan monolitik yazılımlardı. Zamanla bulut bilişimin yaygınlaşmasıyla birlikte, Hizmet Olarak Yazılım (SaaS) modeliyle sunulan bulut tabanlı ERP çözümleri öne çıkmaya başlamıştır.

Bulut tabanlı ERP çözümlerinin temelinde, bu çalışmanın da merkezinde yer alan çok kiracılı mimari bulunur. Bu mimaride, çok sayıda müşteri (kiracı) aynı yazılım örneğini ve altyapıyı paylaşır; ancak her kiracının verisi mantıksal olarak diğerlerinden yalıtılmıştır. Bu yaklaşım, kaynak kullanımını optimize ederek hizmet sağlayıcının ölçek ekonomisinden yararlanmasını sağlar.

Mevcut açık kaynaklı ERP projeleri (örneğin Odoo, ERPNext, Dolibarr) incelendiğinde, bunların güçlü işlevsellik sundukları ancak çoğunlukla olgunlaşmış, karmaşık ve öğrenme eğrisi yüksek sistemler oldukları görülmektedir. Bu çalışmada geliştirilen UniversalERP, ticari bir ürünle rekabet etmekten çok, modern bir teknoloji yığını (ASP.NET Core ve

Vue.js) üzerine, eğitim ve gösterim amacıyla, mimari prensiplerin uygulanmasına odaklanan özgün bir gerçekleştirmedir.

1.4. Çalışmanın Kapsamı ve Sınırlılıkları

Çalışma kapsamında geliştirilen sistem; kullanıcı kimlik doğrulama ve yetkilendirme, kontrol paneli (dashboard), ürün yönetimi, müşteri yönetimi, satış yönetimi, stok takibi, firma (kiracı) ayarları, denetim günlüğü ve temel raporlama modüllerini içermektedir. Sistem, hem sunucu tarafı API'si hem de bu API'yi tüketen tam işlevsel bir tek sayfa uygulaması (SPA) olarak gerçekleştirilmiştir.

Çalışmanın sınırlılıkları ise şu şekildedir: Sistem, gerçek bir ödeme altyapısı (sanal POS) entegrasyonu içermemektedir; satış işlemlerindeki ödeme adımı kavramsal düzeyde ele alınmıştır. Ayrıca e-fatura entegrasyonu, gelişmiş insan kaynakları bordro hesaplamaları ve yapay zekâ destekli tahminleme gibi ileri düzey kurumsal özellikler bu çalışmanın kapsamı dışında bırakılmış, gelecek çalışmalar bölümünde değerlendirilmiştir.

1.5. Tezin Organizasyonu

Bu tez sekiz ana bölümden oluşmaktadır. Birinci bölümde problem tanımlanmış, çalışmanın amacı ve kapsamı ortaya konmuştur. İkinci bölümde projede kullanılan teknolojiler ve geliştirme yöntemleri tanıtılmaktadır. Üçüncü bölüm, sistemin yazılım mimarisini ve tasarım kararlarını detaylandırır. Dördüncü bölümde veritabanı tasarımı ele alınır. Beşinci bölüm, uygulamanın modül modül gerçekleştirilmesini ekran görüntüleriyle birlikte sunar. Altıncı bölüm güvenlik ve yetkilendirme mekanizmalarına ayrılmıştır. Yedinci bölümde test ve performans değerlendirmesi yapılır. Son olarak sekizinci bölümde elde edilen sonuçlar tartışılır ve gelecek çalışmalara yönelik öneriler sunulur.

1.6. ERP Sistemlerinin Tarihsel Gelişimi

Bugün bildiğimiz biçimiyle ERP sistemleri, onlarca yıllık bir evrimin sonucudur. Bu evrimin kökleri, 1960'lı yıllarda üretim işletmelerinin stok kontrolü için geliştirdiği erken bilgisayar sistemlerine dayanır. 1970'lerde bu sistemler, üretim için gerekli malzeme ihtiyacını planlayan Malzeme İhtiyaç Planlaması (MRP) sistemlerine dönüşmüştür. 1980'lerde MRP, üretim kapasitesi, iş gücü ve finansal planlamayı da kapsayacak biçimde genişleyerek Üretim Kaynak Planlaması (MRP II) adını almıştır.

1990'lı yıllara gelindiğinde, bu sistemler yalnızca üretimle sınırlı kalmayıp; insan kaynakları, muhasebe, satış ve dağıtım gibi bir işletmenin tüm temel fonksiyonlarını tek bir bütünleşik platformda toplayacak biçimde olgunlaşmıştır. Bu kapsamlı yapıya, sektörü tanımlayan araştırma kuruluşları tarafından Kurumsal Kaynak Planlama (ERP) adı verilmiştir. Bu dönemde ERP, büyük kurumların milyonlarca dolarlık yatırımlarla kurduğu, kendi veri merkezlerinde barındırdığı dev yazılım sistemleriydi.

2000'li yıllardan itibaren internetin ve bulut bilişimin yaygınlaşması, ERP dünyasında köklü bir dönüşüm başlatmıştır. Hizmet Olarak Yazılım (SaaS) modeli sayesinde, işletmeler artık ERP sistemini kendi sunucularına kurmak yerine, internet üzerinden bir abonelik hizmeti olarak kullanabilir hale gelmiştir. Bu dönüşüm, ERP'yi yalnızca dev kurumların erişebildiği bir lüks olmaktan çıkarıp, çok daha geniş bir işletme yelpazesi için erişilebilir kılmıştır. UniversalERP, bu modern, bulut tabanlı ve çok kiracılı SaaS yaklaşımının somut bir örneği olarak konumlanmaktadır.

1.7. Çok Kiracılığın İş Modeline Katkısı

Çok kiracılı mimarinin önemi yalnızca teknik bir tercih olmasından değil, doğrudan bir iş modelini mümkün kılmasından kaynaklanır. Geleneksel yazılım dağıtımında, her müşteri için yazılımın ayrı bir kopyası kurulur ve bakımı yapılır. Bu, müşteri sayısı arttıkça yönetim maliyetinin doğrusal olarak artması anlamına gelir. Çok kiracılı mimaride ise tek bir yazılım örneği tüm müşterilere hizmet ettiğinden, yeni bir müşterinin sisteme eklenmesinin marjinal maliyeti son derece düşüktür.

Bu durum, ölçek ekonomisi yaratır. Hizmet sağlayıcı, altyapı ve bakım maliyetlerini tüm kiracılara yayarak birim maliyeti düşürür ve bu tasarrufu rekabetçi fiyatlarla son kullanıcıya yansıtabilir. Aynı zamanda, yazılıma eklenen her yeni özellik veya güvenlik güncellemesi, tek bir dağıtımla tüm kiracılara anında ulaşır. Bu merkezi yönetim avantajı, hem hizmet

sağlayıcı hem de kullanıcılar için önemli faydalar sağlar. UniversalERP'nin çok kiracılı tasarımı, bu sürdürülebilir iş modelinin teknik temelini oluşturmaktadır.

2. KULLANILAN TEKNOLOJİLER VE YÖNTEMLER

Bir yazılım projesinin başarısı, doğru teknoloji seçimleriyle yakından ilişkilidir. UniversalERP'nin geliştirilmesinde, sektörde yaygın olarak kullanılan, güçlü topluluk desteğine sahip, performanslı ve modern teknolojiler tercih edilmiştir. Bu bölümde, projenin sunucu (backend) ve istemci (frontend) katmanlarında kullanılan teknolojiler, veritabanı yönetim sistemi, yardımcı kütüphaneler ve benimsenen geliştirme metodolojisi tanıtılmaktadır.

2.1. Backend Teknolojileri

2.1.1. ASP.NET Core 8

Sistemin sunucu tarafı, Microsoft tarafından geliştirilen açık kaynaklı, çapraz platform (cross-platform) bir web çatısı olan ASP.NET Core 8 üzerine inşa edilmiştir. ASP.NET Core, yüksek performansı, dahili bağımlılık enjeksiyonu (dependency injection) desteği ve ara katman (middleware) tabanlı esnek istek işleme hattıyla modern web API'leri geliştirmek için güçlü bir temel sunar. .NET 8'in Uzun Vadeli Destek (LTS) sürümü olması, projenin uzun süreli sürdürülebilirliği açısından önemli bir tercih nedenidir.

ASP.NET Core'un sunduğu denetleyici (controller) tabanlı yapı sayesinde, RESTful API uç noktaları temiz ve düzenli bir biçimde tanımlanmıştır. Asenkron programlama (async/await) modelinin baştan sona kullanılması, sunucunun aynı anda çok sayıda isteği bloke olmadan karşılayabilmesini ve böylece yüksek ölçeklenebilirlik sağlanmasını mümkün kılmıştır.

2.1.2. Entity Framework Core

Veritabanı işlemleri, Microsoft'un nesne-ilişkisel eşleme (ORM) aracı olan Entity Framework Core (EF Core) ile yönetilmiştir. EF Core, geliştiricinin C# sınıfları üzerinden veritabanı tablolarıyla çalışmasına olanak tanır; SQL sorgularını LINQ ifadeleri biçiminde yazmayı ve bunların otomatik olarak optimize edilmiş SQL'e çevrilmesini sağlar. Bu yaklaşım, hem geliştirme hızını artırır hem de SQL enjeksiyonu gibi güvenlik açıklarına karşı dahili koruma sunar.

EF Core'un bu projede kritik öneme sahip iki özelliği vardır. Birincisi, veritabanı şemasının kod üzerinden sürüm kontrollü biçimde yönetilmesini sağlayan göç (migration) mekanizmasıdır. İkincisi ise çok kiracılı mimarının kalbinde yer alan ve her sorguya otomatik olarak kiracı filtresi ekleyen Global Query Filters özelliğidir.

Çizelge 2.1. Kullanılan backend teknolojileri ve sürümleri

Teknoloji / Kütüphane	Sürüm	Kullanım Amacı
ASP.NET Core	8.0	Web API çatısı
Entity Framework Core	8.0	ORM ve veritabanı erişimi
AutoMapper	12.x	Varlık-DTO dönüşümleri
FluentValidation	11.x	İstek doğrulama kuralları
JWT Bearer	8.0	Kimlik doğrulama
BCrypt.Net	4.x	Parola özetleme (hashing)
Serilog	3.x	Yapılandırılmış günlükleme
Swashbuckle (Swagger)	6.x	API dokümantasyonu

2.2. Frontend Teknolojileri

2.2.1. Vue.js 3

Kullanıcı arayüzü, ilerlemeli (progressive) bir JavaScript çatısı olan Vue.js 3 kullanılarak geliştirilmiştir. Vue.js, tepkisel (reactive) veri bağlama modeli ve bileşen (component) tabanlı mimarisi sayesinde, karmaşık ve etkileşimli kullanıcı arayüzlerinin yönetilebilir parçalara bölünerek geliştirilmesini sağlar. Vue 3 ile gelen Composition API, bileşen mantığının daha modüler ve yeniden kullanılabilir biçimde organize edilmesine olanak tanımıştır.

Uygulama, tek sayfa uygulaması (SPA) olarak tasarlanmıştır. Bu sayede, kullanıcı modüller arasında geçiş yaparken sayfanın tamamen yeniden yüklenmesi yerine yalnızca gerekli veri parçaları sunucudan asenkron olarak çekilir. Bu durum, masaüstü uygulamalarına yakın, akıcı ve hızlı bir kullanıcı deneyimi sunar.

2.2.2. Pinia ve Vue Router

Uygulama genelinde paylaşılan durumun (state) yönetimi için Vue ekosisteminin resmi durum yönetim kütüphanesi olan Pinia kullanılmıştır. Oturum açmış kullanıcının bilgileri, kimlik doğrulama jetonu ve modüller arası paylaşılan veriler, Pinia depoları (store) aracılığıyla merkezi ve tutarlı biçimde yönetilmektedir. Sayfalar arası yönlendirme ise Vue Router ile sağlanmış; yönlendirme korumaları (route guards) sayesinde yetkisiz erişimler giriş ekranına yönlendirilmektedir.

Çizelge 2.2. Kullanılan frontend teknolojileri ve sürümleri

Teknoloji / Kütüphane	Sürüm	Kullanım Amacı
Vue.js	3.x	Kullanıcı arayüzü çatısı

Teknoloji / Kütüphane	Sürüm	Kullanım Amacı
Vite	5.x	Geliştirme ve derleme aracı
Pinia	2.x	Durum (state) yönetimi
Vue Router	4.x	İstemci tarafı yönlendirme
Axios	1.x	HTTP istemcisi (API tüketimi)

2.3. Veritabanı Yönetim Sistemi

Verilerin kalıcı olarak saklanması için ilişkisel bir veritabanı yönetim sistemi olan Microsoft SQL Server tercih edilmiştir. SQL Server'ın ACID (Atomiklik, Tutarlılık, İzolasyon, Dayanıklılık) ilkelerine tam uyumu, gelişmiş indeksleme yetenekleri ve .NET ekosistemiyle olan doğal entegrasyonu, bu tercihte belirleyici olmuştur. Veritabanı şeması, Entity Framework Core göçleri aracılığıyla oluşturulmuş ve yönetilmiştir.

Çok kiracılı yapının gereği olarak, tüm kiracılara ait veriler aynı veritabanında, ortak tablolarda saklanmakta; ancak her kayıt bir kiracı kimliği (TenantId) ile etiketlenmektedir. Bu satır düzeyinde izolasyon (row-level isolation) yaklaşımı, kiracı başına ayrı veritabanı oluşturmanın getireceği yönetim karmaşıklığını ortadan kaldırırken, Global Query Filters sayesinde güvenli bir veri ayrıştırması sağlar.

2.4. Yardımcı Kütüphaneler ve Araçlar

Projede temel çatıların yanı sıra geliştirme verimliliğini ve kod kalitesini artıran çeşitli yardımcı kütüphaneler kullanılmıştır. Varlık nesneleri ile veri transfer nesneleri (DTO) arasındaki dönüşümler AutoMapper ile otomatikleştirilmiş; gelen isteklerin doğrulanması FluentValidation ile iş mantığından ayrıştırılmıştır. Parolalar, geri döndürülemez biçimde BCrypt algoritmasıyla özetlenerek saklanmaktadır. Sistem genelindeki olaylar ve hatalar, yapılandırılmış günlükleme kütüphanesi Serilog ile kaydedilmektedir. API'nin dokümantasyonu ve test edilebilmesi için Swagger/OpenAPI arayüzü entegre edilmiştir.

2.5. Geliştirme Metodolojisi

Proje, çevik (agile) yazılım geliştirme prensipleri benimsenerek, yinelemeli (iterative) ve artımlı (incremental) bir yaklaşımla geliştirilmiştir. Geliştirme süreci, her biri belirli bir modülün veya özelliğin tamamlanmasına odaklanan kısa yinelemelere bölünmüştür. Önce çekirdek mimari ve kimlik doğrulama altyapısı kurulmuş, ardından ürün, müşteri ve satış modülleri sırayla eklenmiştir.

Kaynak kodu yönetimi için Git sürüm kontrol sistemi kullanılmış; kod deposu GitHub üzerinde barındırılmıştır. Her anlamlı değişiklik, açıklayıcı işleme (commit) mesajlarıyla kayıt altına alınarak projenin gelişim geçmişi izlenebilir kılınmıştır. Bu metodoloji, geliştirme sürecinde ortaya çıkan gereksinim değişikliklerine esnek biçimde uyum sağlanmasına olanak tanımıştır.

2.6. REST Mimari Stili ve API Tasarım İlkeleri

UniversalERP'nin sunucu ve istemci katmanları arasındaki iletişim, Roy Fielding tarafından doktora tezinde tanımlanan REST (Representational State Transfer) mimari stiline uygun biçimde tasarlanmıştır. REST, web'in temel protokolü olan HTTP'nin doğal yeteneklerinden yararlanan, durumsuz ve kaynak odaklı bir iletişim yaklaşımıdır. Bu yaklaşımda her iş varlığı (müşteri, ürün, satış) bir kaynak olarak ele alınır ve benzersiz bir adres (URI) ile temsil edilir.

Kaynaklar üzerindeki işlemler, HTTP'nin standart metodlarıyla ifade edilir. Bir kaynağı okumak için GET, yeni bir kaynak oluşturmak için POST, mevcut bir kaynağı güncellemek için PUT, silmek için ise DELETE metodu kullanılır. Bu standartlaşma, API'nin öngörülebilir ve kolay anlaşılır olmasını sağlar. İşlemin sonucu, istemciye anlamlı HTTP durum kodlarıyla bildirilir: başarılı bir okuma için 200, başarılı bir oluşturma için 201, yetkisiz erişim için 401, bulunamayan kaynak için 404 gibi.

API'nin tutarlılığı açısından, tüm yanıtlar standart bir yapı içinde döndürülür. Başarılı yanıtlar talep edilen veriyi taşıırken, hatalı durumlarda istemciye hatanın türünü ve açıklamasını içeren yapılandırılmış bir hata nesnesi iletilir. Bu tutarlı yapı, istemci tarafında hata yönetiminin merkezi ve güvenilir biçimde yapılmasına olanak tanır. Ayrıca tüm liste uç noktaları, büyük veri kümelerinin verimli biçimde aktarılması için sayfalama (pagination), filtreleme ve sıralama parametrelerini destekler.

2.7. Asenkron Programlama Modeli

Modern web uygulamalarının yüksek sayıda eş zamanlı isteği verimli biçimde karşılayabilmesi, asenkron programlama modelinin etkin kullanımına bağlıdır. Geleneksel senkron (eş zamanlı) modelde, bir işlem (örneğin bir veritabanı sorgusu) tamamlanana kadar ilgili sunucu iş parçacığı (thread) bloke olur ve başka bir iş yapamaz. Veritabanı veya ağ işlemleri gibi giriş/çıkış (I/O) ağırlıklı görevlerde bu bekleme, sunucu kaynaklarının verimsiz kullanılmasına yol açar.

UniversalERP, baştan sona C#'ın async/await asenkron programlama modeli kullanılarak geliştirilmiştir. Bu modelde, bir iş parçacığı bir I/O işleminin tamamlanmasını beklerken bloke olmak yerine serbest bırakılır ve bu sırada başka istekleri işleyebilir. İşlem tamamlandığında, yürütme kaldığı yerden devam eder. Bu yaklaşım, aynı donanım kaynaklarıyla çok daha fazla eş zamanlı isteğin karşılanmasını sağlayarak sistemin verimini ve ölçeklenebilirliğini önemli ölçüde artırır.

Veri erişim katmanındaki tüm depo metotları (GetByIdAsync, GetAllAsync, AddAsync) ve bunları çağıran servis metotları asenkron olarak tasarlanmıştır. Bu tutarlı asenkron zincir, isteğin denetleyiciden veritabanına kadar olan tüm yolculuğunun bloke olmadan ilerlemesini güvence altına alır. Yedinci bölümde sunulan performans sonuçlarında gözlemlenen yüksek eş zamanlı kullanıcı kapasitesinin arkasında, büyük ölçüde bu asenkron tasarım yatmaktadır.

3. SİSTEM MİMARİSİ VE TASARIMI

Bir yazılım sisteminin uzun vadeli başarısı, yalnızca işlevlerini doğru yerine getirmesiyle değil; aynı zamanda ne kadar kolay anlaşılabilirliği, test edilebilirliği, bakımı yapılabilirliği ve genişletilebilirliğiyle ölçülür. Bu nitelikler, doğrudan yazılımın mimarisiyle ilişkilidir. UniversalERP'nin geliştirilmesinde, bu kalite niteliklerini en üst düzeye çıkarmayı hedefleyen Clean Architecture yaklaşımı benimsenmiştir. Bu bölümde, sistemin mimari tasarımı, uygulanan tasarım desenleri ve çok kiracılı yapının nasıl gerçekleştirildiği detaylandırılmaktadır.

3.1. Clean Architecture Yaklaşımı

Robert C. Martin tarafından öne sürülen Clean Architecture, yazılımı iç içe geçmiş eşmerkezli katmanlar biçiminde ele alan bir mimari felsefedir. Bu felsefenin merkezinde Bağımlılık Kuralı (The Dependency Rule) yer alır: kaynak kod bağımlılıkları yalnızca içe doğru, yani dış katmanlardan iç katmanlara doğru olabilir. İç katmanlar, kendilerini çevreleyen dış katmanların varlığından dahi habersizdir. Bu kural sayesinde, iş kurallarını barındıran çekirdek katman; veritabanı, web çatısı veya kullanıcı arayüzü gibi dışsal ve değişken ayrıntılardan tamamen yalıtılır.

Bu yalıtımın pratik faydası büyüktür. Örneğin, gelecekte veritabanının SQL Server yerine PostgreSQL ile değiştirilmesi gerektiğinde, yalnızca en dıştaki veri erişim katmanı değiştirilir; iş kurallarını içeren çekirdek katmana hiç dokunulmaz. Aynı şekilde iş mantığı, herhangi bir web sunucusuna ihtiyaç duymadan birim testlerinde sınanabilir. UniversalERP, bu prensip doğrultusunda üç ana projeye ayrılmıştır:

- ERP.Core (Çekirdek Katman): Varlıkları (entities), veri transfer nesnelerini (DTO), arayüzleri (interfaces) ve eşleme profillerini içerir. Hiçbir dış katmana bağımlılığı yoktur. Sistemin kalbidir.
- ERP.Data (Veri Katmanı): Çekirdek katmanda tanımlanan arayüzlerin somut gerçekleştirmelerini barındırır. Entity Framework Core DbContext'i, Repository ve Unit of Work gerçekleştirmeleri ile iş mantığını yürüten servisler buradadır.
- ERP.API (Sunum Katmanı): Dış dünyaya açılan kapıdır. HTTP isteklerini karşılayan denetleyiciler (controllers), ara katmanlar (middleware), kimlik doğrulama yapılandırması ve SignalR merkezi (hub) bu katmanda yer alır.

Bu üçlü yapıya ek olarak, sistemin doğruluğunu güvence altına alan birim ve entegrasyon testlerini içeren ayrı bir ERP.Tests projesi de bulunmaktadır. Bu ayırım sayesinde, her

katmanın sorumluluğu açıkça tanımlanmış ve katmanlar arası bağımlılıklar denetim altına alınmıştır.

3.2. Katmanlı Mimari Yapısı

Clean Architecture'in kavramsal katmanları, çalışma zamanında somut bir istek işleme akışına dönüşür. Bir kullanıcının arayüzde gerçekleştirdiği bir eylem (örneğin yeni bir müşteri eklemesi), aşağıdaki katmanlardan sırayla geçer:

7. Sunum Katmanı (Presentation): Vue.js arayüzü, kullanıcıdan aldığı veriyi bir HTTP isteğine dönüştürerek API'ye gönderir.
8. API Katmanı (Controllers): İlgili denetleyici isteği karşılar, gelen veriyi FluentValidation ile doğrular ve iş mantığı katmanını çağırır.
9. İş Mantığı Katmanı (Services): Servis sınıfı, iş kurallarını uygular. Örneğin, bir satış işleminde stok yeterliliğini denetler, denetim günlüğü kaydı oluşturur.
10. Veri Erişim Katmanı (Repository / Unit of Work): Servis, veritabanı işlemlerini doğrudan değil, Repository soyutlaması üzerinden gerçekleştirir. Unit of Work, ilgili tüm değişikliklerin tek bir işlem (transaction) içinde tutarlı biçimde kaydedilmesini sağlar.
11. Veritabanı Katmanı: Entity Framework Core, LINQ sorgularını SQL'e çevirerek SQL Server üzerinde yürütür ve sonucu geri döndürür.

Bu tek yönlü ve katmanlı akış, her katmanın yalnızca kendi sorumluluğuna odaklanmasını sağlar. Sunum katmanı verinin nasıl saklandığını bilmez; veri katmanı verinin nasıl gösterileceğiyle ilgilenmez. Bu görev ayrımı (separation of concerns), kodun anlaşılabilirliğini ve bakımını önemli ölçüde kolaylaştırır.

3.3. Tasarım Desenleri

Sistemin mimarisi, kanıtlanmış çeşitli tasarım desenleri (design patterns) üzerine kurulmuştur. Bu desenler, sık karşılaşılan tasarım problemlerine kanıtlanmış çözümler sunarak kodun kalitesini ve tutarlılığını artırır.

3.3.1. Repository (Depo) Deseni

Repository deseni, veri erişim mantığını iş mantığından soyutlar. İş katmanındaki servisler, veriye erişmek için doğrudan DbContext ile değil, genel bir IRepository<T> arayüzü üzerinden çalışır. Bu sayede veri erişim teknolojisi değişse dahi iş mantığı etkilenmez ve

servisler test edilirken sahte (mock) depolar kullanılabilir. Aşağıda genel deponun arayüz tanımını yer almaktadır:

```
public interface IRepository<T> where T : BaseEntity
{
    Task<T> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
}
```

3.3.2. Unit of Work (İş Birimi) Deseni

Unit of Work deseni, mantıksal olarak tek bir iş sayılan birden çok veritabanı işleminin, ya tümüyle başarılı olmasını ya da tümüyle geri alınmasını güvence altına alır. Örneğin bir satış kaydedilirken hem satış tablosuna kayıt eklenir hem ürün stoğu güncellenir hem de denetim günlüğüne bir satır yazılır. Unit of Work, bu üç işlemin tek bir veritabanı işleminde (transaction) atomik biçimde gerçekleştirilmesini sağlayarak veri tutarlılığını korur.

3.3.3. Bağımlılık Enjeksiyonu (Dependency Injection)

ASP.NET Core'un dahili bağımlılık enjeksiyonu konteyneri, sınıflar arası bağımlılıkların yönetiminde merkezi rol oynar. Servisler, depolar ve DbContext, uygulama başlangıcında konteynere kaydedilir; ihtiyaç duyan sınıflara yapıcı metot (constructor) aracılığıyla otomatik olarak sağlanır. Bu yaklaşım, sınıfların somut gerçekleştirmelere değil soyutlamalara bağlı kalmasını sağlayarak gevşek bağımlılık (loose coupling) oluşturur.

3.3.4. Eşleme (Mapper) Deseni

Veritabanı varlıklarının (entity) doğrudan dış dünyaya açılması hem güvenlik hem de tasarım açısından sakıncalıdır. Bu nedenle istemciyle veri alışverişinde veri transfer nesneleri (DTO) kullanılır. Varlıklar ile DTO'lar arasındaki dönüşümler, tekrarlayan kod yazımını önlemek için AutoMapper kütüphanesiyle otomatikleştirilmiştir.

3.4. SOLID Prensiplerinin Uygulanması

SOLID, nesne yönelimli tasarımın beş temel prensibinin kısaltmasıdır ve sürdürülebilir yazılımın temelini oluşturur. UniversalERP'nin tasarımında bu prensiplere özen gösterilmiştir:

- Tek Sorumluluk Prensipleri (SRP): Her sınıfın değişmesi için tek bir nedeni vardır. Örneğin CustomerService yalnızca müşteriyle ilgili iş mantığından sorumludur; satış işlemleri ayrı bir SaleService tarafından yürütülür.

- Açık/Kapalı Prensibi (OCP): Sınıflar geliştirmeye açık, değiştirmeye kapalıdır. Yeni bir varlık türü için mevcut kodu değiştirmeden, genel depo yapısı üzerinden yeni gerçekleştirmeler eklenebilir.
- Liskov Yerine Geçme Prensibi (LSP): Bir arayüzü gerçekleştiren tüm somut sınıflar, o arayüzün beklenen davranışını bozmadan birbirinin yerine kullanılabilir.
- Arayüz Ayrımı Prensibi (ISP): Geniş ve çok amaçlı arayüzler yerine, istemcilerin yalnızca ihtiyaç duyduğu metotları içeren küçük ve özelleşmiş arayüzler tercih edilmiştir.
- Bağımlılığın Ters Çevrilmesi Prensibi (DIP): Üst düzey modüller, alt düzey modüllere değil; her ikisi de soyutlamalara bağımlıdır. Servisler somut depo sınıflarına değil, IRepository arayüzüne bağlıdır.

3.5. Çok Kiracılı (Multi-Tenant) Mimari

Bu çalışmanın en ayırt edici özelliği, çok kiracılı mimarisidir. Çok kiracılılık, tek bir yazılım örneğinin birden çok müşteriye (kiracıya) hizmet verdiği; ancak her kiracının verisinin diğerlerinden yalıtıldığı bir yazılım mimarisi yaklaşımıdır. UniversalERP'de, kiracı başına ayrı veritabanı oluşturmak yerine, tüm kiracıların aynı veritabanını paylaştığı ancak verilerin satır düzeyinde ayrıştırıldığı paylaşımlı veritabanı yaklaşımı benimsenmiştir.

Bu yaklaşımda, kiracıya özgü tüm varlıklar bir TenantId alanı içerir. Bir kullanıcı sisteme giriş yaptığında, ait olduğu kiracının kimliği üretilen JWT jetonunun içine bir talep (claim) olarak gömülür. Sonraki her istekte, bu kimlik jetondan okunarak isteğin bağlamına yerleştirilir. İşin kritik kısmı ise verinin nasıl filtrelendiğidir.

Eğer her sorguya kiracı filtresini eklemek geliştiriciye bırakılsaydı, unutulmuş tek bir filtre dahi bir kiracının verisinin başka bir kiracıya sızmasına yol açabilirdi. Bu riski tamamen ortadan kaldırmak için Entity Framework Core'un Global Query Filters özelliği kullanılmıştır. Veritabanı bağlamında (DbContext) bir kez tanımlanan bu filtre, ilgili varlığa yönelik istisnasız her sorguya, geliştirici hiçbir şey yazmasa dahi, otomatik olarak aktif kiracı koşulunu ekler.

Aşağıdaki kod parçası, bu filtrenin nasıl tanımlandığını göstermektedir. Bu sayede güvenli veri izolasyonu, geliştiricinin dikkatine bırakılmak yerine altyapının bir garantisi hâline getirilmiştir:

```
modelBuilder.Entity<Customer>()
    .HasQueryFilter(c => c.TenantId == _tenantProvider.GetTenantId());
```

```
modelBuilder.Entity<Product>()
    .HasQueryFilter(p => p.TenantId == _tenantProvider.GetTenantId());
```

Bu mimarinin bir diğer avantajı, en üst düzey yetkiye sahip Super Admin rolünün, sistem yönetimi amacıyla kiracılar arası işlem yapılabilmesine olanak tanıyacak biçimde tasarlanabilmesidir. Çizelge 3.1, sistemde tanımlı rolleri ve yetki düzeylerini özetlemektedir.

Çizelge 3.1. Sistemdeki kullanıcı rolleri ve yetki düzeyleri

Rol	Yetki Düzeyi ve Kapsamı
Super Admin	Tüm kiracıları ve sistem ayarlarını yönetir. En üst düzey yetki.
Admin	Kendi kiracısı içinde tam yetkilidir; tüm modüllere erişir.
Manager	Satış, müşteri ve raporlama modüllerini yönetir.
Staff	Satış ve servis kaydı girer; listeleri görüntüler.
Viewer	Yalnızca okuma yetkisine sahiptir; veri giremez.
Guest	En sınırlı roldür; yalnızca temel paneli görüntüler.

3.6. Çapraz Kesen İlgiler ve Ara Katman Mimarisi

Bir uygulamada günlüklenme, hata yönetimi, kimlik doğrulama ve denetim kaydı gibi bazı işlevler, neredeyse her isteği ilgilendirir. Bu tür, sistemin birçok noktasını dikey olarak kesen işlevlere çapraz kesen ilgiler (cross-cutting concerns) denir. Bu işlevlerin her uç noktada tekrar tekrar yazılması, kod tekrarına ve bakım zorluğuna yol açar. UniversalERP, bu sorunu ASP.NET Core'un ara katman (middleware) hattı sayesinde zarif biçimde çözer.

Ara katmanlar, bir HTTP isteğinin denetleyiciye ulaşmadan önce ve yanıtın istemciye dönmenden önce geçtiği bir işlem zinciri oluşturur. UniversalERP'de, merkezi bir istisna yönetimi ara katmanı (ExceptionMiddleware) tanımlanmıştır. Bu ara katman, uygulamanın herhangi bir noktasında oluşan beklenmeyen hataları yakalar, bunları günlüğe kaydeder ve istemciye, sistemin iç detaylarını sızdırmayan, standart ve güvenli bir hata yanıtı döndürür. Böylece hata yönetimi mantığı tek bir yerde toplanmış olur.

Benzer biçimde, kimlik doğrulama ve kiracı bağlamının belirlenmesi de ara katmanlar aracılığıyla gerçekleştirilir. Her istek, denetleyiciye ulaşmadan önce kimlik doğrulama ara katmanından geçer; burada JWT jetonu doğrulanır ve içindeki kiracı kimliği uygulama bağlamına yerleştirilir. Bu sayede, iş mantığı katmanındaki kod, kimlik doğrulama ve kiracı belirleme ayrıntılarıyla hiç ilgilenmeden, yalnızca kendi sorumluluğuna odaklanabilir.

3.7. Bağımlılıkların Yapılandırılması ve Servis Yaşam Döngüleri

Bağımlılık enjeksiyonu konteynerine kaydedilen her servisin bir yaşam döngüsü (lifetime) vardır ve bu yaşam döngüsünün doğru seçilmesi sistemin doğruluğu açısından kritiktir. ASP.NET Core üç temel yaşam döngüsü sunar: her istek için yeni bir örnek oluşturan Scoped, uygulama boyunca tek bir örnek paylaşan Singleton ve her ihtiyaçta yeni örnek üreten Transient.

UniversalERP'de, veritabanı bağlamı (DbContext) ve depolar Scoped olarak kaydedilmiştir; çünkü bir veritabanı işleminin bütünlüğü tek bir istek bağlamında korunmalıdır. Aynı istek içindeki tüm servislerin aynı DbContext örneğini paylaşması, Unit of Work deseninin doğru çalışması için gereklidir. Kiracı bilgisini sağlayan servis de istek bağlamına bağlı olduğundan Scoped olarak yapılandırılmıştır. Bu yaşam döngüsü kararları, çok kiracılı yapının her istekte doğru kiracı bağlamıyla çalışmasını güvence altına alır.

4. VERİTABANI TASARIMI

Veritabanı, bir ERP sisteminin en değerli varlığı olan veriyi barındıran temeldir. İyi tasarlanmış bir veritabanı şeması; veri bütünlüğünü korur, tekrarı önler ve sorguların verimli çalışmasını sağlar. Bu bölümde UniversalERP'nin veritabanı tasarımı, varlıklar arası ilişkiler, kiracı izolasyonu ve performans optimizasyonları ele alınmaktadır.

4.1. Varlık-İlişki Modeli

Sistemin veri modeli, gerçek dünyadaki iş varlıklarını ve aralarındaki ilişkileri yansıtacak biçimde tasarlanmıştır. Tüm kiracıya özgü varlıklar, ortak alanları (kimlik, oluşturulma tarihi, kiracı kimliği, aktiflik durumu) barındıran bir BaseEntity temel sınıfından türetilmiştir. Bu yaklaşım, kod tekrarını azaltır ve denetim günlüğü gibi çapraz kesen ilgileri merkezi biçimde yönetmeyi kolaylaştırır.

Modeldeki temel varlıklar ve ilişkileri şu şekilde özetlenebilir: Bir Tenant (kiracı/firma), birçok AppUser (kullanıcı), Customer (müşteri), Product (ürün) ve Sale (satış) kaydına sahiptir. Bir Customer, birden çok Sale ile ilişkilidir. Bir Sale, bir Product ve bir Customer'a referans verir. Her veri değişikliği, ilgili kaydı ve değişikliği yapan kullanıcıyı işaret eden bir AuditLog kaydı üretir.

4.2. Tablo Yapıları ve İlişkiler

Veritabanı şeması, ilişkisel veritabanı tasarımının üçüncü normal forma (3NF) uygun biçimde normalleştirilmiştir. Bu, veri tekrarını en aza indirir ve güncelleme anomalilerini önler. Tablolar arası ilişkiler, yabancı anahtar (foreign key) kısıtlamalarıyla güvence altına alınarak referans bütünlüğü korunmuştur. Çizelge 4.1, sistemin temel tablolarını ve işlevlerini özetlemektedir.

Çizelge 4.1. Temel veritabanı tabloları ve açıklamaları

Tablo	Açıklama
Tenants	Sisteme kayıtlı firmaları (kiracıları) tutar.
AppUsers	Kullanıcı hesaplarını, parola özetlerini ve rollerini saklar.
Customers	Her kiracıya ait müşteri kayıtlarını içerir.
Products	Ürün kataloğunu, fiyat ve stok bilgisini barındırır.
Sales	Satış işlemlerini ve durum bilgisini kaydeder.
ServiceOrders	Servis/hizmet siparişlerini ve durumlarını yönetir.
ProjectTasks	Görev ve proje takibi kayıtlarını tutar.
AuditLogs	Tüm kritik veri değişikliklerinin denetim kaydını tutar.

Her tabloda, kaydın hangi kiracıya ait olduğunu belirten TenantId alanı bulunur. Ayrıca silme işlemleri, kayıtların veritabanından fiziksel olarak silinmesi yerine, IsActive bayrağının pasife çekildiği yumuşak silme (soft delete) yaklaşımıyla gerçekleştirilir. Bu sayede yanlışlıkla silinen verilerin kurtarılması mümkün olur ve geçmiş kayıtların bütünlüğü korunur.

4.3. Veri İzolasyonu ve Global Query Filters

Üçüncü bölümde mimari açıdan ele alınan çok kiracılı veri izolasyonu, veritabanı düzeyinde Global Query Filters mekanizmasıyla hayata geçirilmiştir. Bu mekanizmanın çalışma mantığı şu adımları izler: Kullanıcı giriş yaptığında kiracı kimliği jetona yazılır; her istekte bir ara katman (middleware) bu kimliği okuyup uygulama bağlamına aktarır; DbContext, bu bağlamdaki kimliği kullanarak tüm sorgulara otomatik kiracı koşulunu ekler.

Bu yaklaşımın en büyük değeri, güvenliği bireysel sorgu yazımının doğruluğuna bağımlı olmaktan çıkarıp altyapısal bir güvenceye dönüştürmesidir. Geliştirici bir sorguda kiracı filtresini açıkça belirtmeyi unutsa bile, filtre arka planda otomatik olarak uygulandığından veri sızıntısı riski ortadan kalkar. İhtiyaç duyulan özel durumlarda (örneğin Super Admin işlemlerinde) filtre, IgnoreQueryFilters() yöntemiyle bilinçli olarak devre dışı bırakılabilir.

4.4. İndeksleme ve Performans Optimizasyonu

Veri hacmi büyüdükçe sorgu performansı kritik hale gelir. UniversalERP'de en sık çalıştırılan sorgu desenleri analiz edilerek, bu desenlere uygun bileşik indeksler (composite indexes) tanımlanmıştır. Çok kiracılı yapıda neredeyse her sorgu TenantId alanıyla filtrelendiğinden, indeksler bu alan önde olacak biçimde tasarlanmıştır.

Örneğin, bir firmanın aktif müşterilerini listeleyen sorgu için oluşturulan (TenantId, IsActive) bileşik indeksi, ilgili sorgunun yanıt süresini önemli ölçüde kısaltmıştır. Benzer biçimde, satışları tarihe göre sıralayan raporlar için (TenantId, SaleDate) indeksi tanımlanmıştır.

Performansı etkileyen bir diğer önemli konu, ORM kullanımında sık rastlanan N+1 sorgu problemidir. Bu problem, bir ana kayıt listesi çekildikten sonra her bir kaydın ilişkili verisi için ayrı bir sorgu çalıştırılması durumunda ortaya çıkar ve veritabanını gereksiz yere yorar. UniversalERP'de bu problem, ilişkili verilerin tek bir sorguda birlikte çekilmesini sağlayan EF Core'un Include() ve projeksiyon (Select) yöntemleriyle çözülmüştür. Ayrıca yalnızca

okuma amaçlı sorgularda AsNoTracking() kullanılarak, gereksiz değişiklik takibinin getirdiği bellek ve işlem maliyeti ortadan kaldırılmıştır.

Sık erişilen ve seyrek değişen veriler (örneğin kontrol panelindeki özet metrikler) için bellek içi önbellekleme (in-memory caching) uygulanmıştır. Bu sayede her panel açılışında veritabanını yeniden sorgulamak yerine, önbellekteki güncel veri sunularak hem yanıt süresi kısaltılmış hem de veritabanı yükü azaltılmıştır.

4.5. Veritabanı Göçleri ve Sürüm Yönetimi

Bir yazılım projesi geliştikçe veritabanı şeması da kaçınılmaz olarak değişir; yeni tablolar eklenir, mevcut tablolara yeni alanlar gelir. Bu değişikliklerin denetimsiz biçimde yapılması, özellikle ekip çalışmasında ve farklı ortamlar (geliştirme, test, üretim) arasında ciddi tutarsızlıklara yol açar. UniversalERP, bu sorunu Entity Framework Core'un göç (migration) mekanizmasıyla çözer.

Göçler, veritabanı şemasındaki her değişikliği, sürüm kontrolüne dahil edilebilen kod dosyaları olarak ifade eder. Bir geliştirici C# varlık sınıflarında değişiklik yaptığında, bu değişikliği veritabanına yansıtacak yeni bir göç dosyası üretilir. Bu dosya, şemayı bir önceki durumdan yeni duruma taşıyan adımları (tablo oluşturma, sütun ekleme, indeks tanımlama) içerir. Göçler kronolojik olarak uygulandığında, herhangi bir ortamdaki veritabanı, kod ile tam uyumlu ve tutarlı bir duruma getirilebilir.

Bu yaklaşım, veritabanı şemasının da tıpkı kaynak kodu gibi sürümlenmesini, gözden geçirilmesini ve gerektiğinde geri alınmasını mümkün kılar. Projenin başlangıç şemasını oluşturan ilk göç (InitialCreate) ile sonraki geliştirme aşamalarında eklenen göçler, veritabanının evrim geçmişini eksiksiz biçimde belgeler. Üretim ortamına dağıtım sırasında, bekleyen göçler otomatik olarak uygulanarak şemanın güncel tutulması sağlanır.

4.6. Veri Bütünlüğü ve Kısıtlamalar

Veritabanı düzeyinde veri bütünlüğünün korunması, uygulamanın güvenilirliği için temel bir gereksinimdir. UniversalERP'nin veritabanı tasarımında, çeşitli kısıtlama türleriyle veri bütünlüğü güvence altına alınmıştır. Birincil anahtar (primary key) kısıtlamaları, her kaydın benzersiz biçimde tanımlanmasını sağlar. Yabancı anahtar (foreign key) kısıtlamaları, ilişkili tablolar arasındaki referansların geçerliliğini korur; örneğin, var olmayan bir müşteriye satış kaydı oluşturulması engellenir.

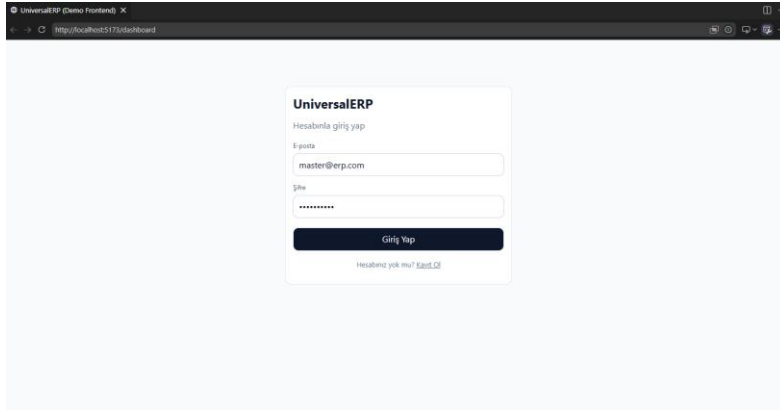
Ayrıca, belirli alanlar üzerinde benzersizlik (unique) kısıtlamaları tanımlanmıştır. Örneğin, aynı kiracı içinde iki kullanıcının aynı e-posta adresine sahip olması engellenir. Bu kısıtlamaların veritabanı düzeyinde tanımlanması, uygulama katmanındaki olası bir hatanın dahi veri bütünlüğünü bozmasını önleyen son bir savunma hattı oluşturur. Böylece veri tutarlılığı, yalnızca uygulama mantığına değil, veritabanının kendi güvencelerine de dayandırılmış olur.

5. UYGULAMANIN GERÇEKLEŞTİRİLMESİ

Bu bölüm, önceki bölümlerde tasarımı ve mimarisi anlatılan UniversalERP sisteminin somut gerçekleştirimini, çalışan uygulamanın ekran görüntüleriyle birlikte modül modül ele almaktadır. Her modül için, sunduğu işlevsellik, arka planda gerçekleşen teknik akış ve kullanıcıya sunulan arayüz açıklanmaktadır. Böylece tasarım kararlarının nihai üründe nasıl karşılık bulduğu somut biçimde gösterilmektedir.

5.1. Kimlik Doğrulama ve Oturum Yönetimi

Kullanıcının sistemle kurduğu ilk temas, kimlik doğrulama ekranıdır. Bu ekran, hem güvenliğin ilk kapısı hem de kullanıcı deneyiminin başlangıç noktasıdır. UniversalERP'nin giriş ekranı, sade ve odaklı bir tasarımla, kullanıcıdan yalnızca e-posta ve parola bilgisini talep eder. Şekil 5.1'de sisteme giriş ekranı görülmektedir.



Şekil 5.1. Sisteme giriş (login) ekranı

Kullanıcı giriş bilgilerini girip Giriş Yap düğmesine bastığında, arka planda şu teknik akış işler: İstemci, girilen bilgileri API'nin kimlik doğrulama uç noktasına gönderir. Sunucu, gelen e-postaya ait kullanıcıyı veritabanında bulur ve girilen parolanın BCrypt özetini, veritabanında saklanan özetle karşılaştırır. Parolaların asla düz metin olarak saklanmadığını, yalnızca geri döndürülemez özetlerinin tutulduğunu vurgulamak gerekir.

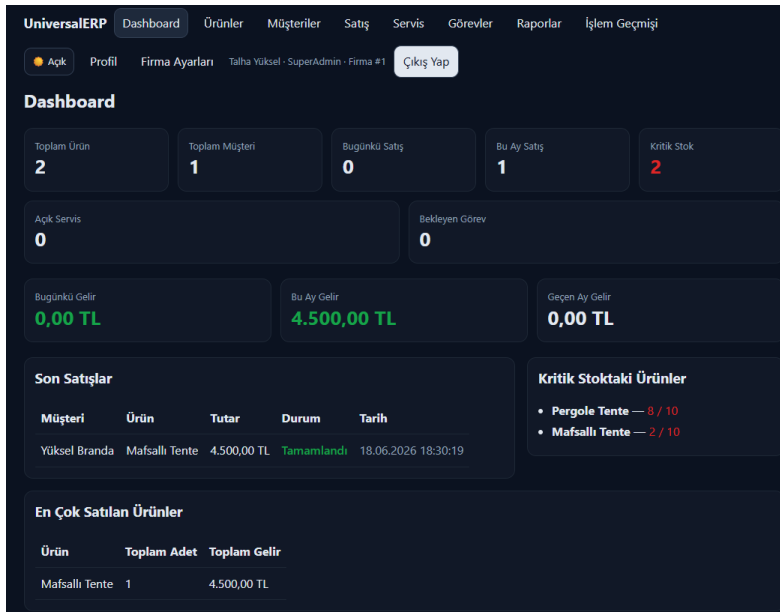
Kimlik bilgileri doğrulandığında sunucu, kullanıcının kimliğini, ait olduğu kiracıyı (TenantId) ve rollerini içeren imzalı bir JSON Web Token (JWT) üretir ve istemciye döndürür. İstemci tarafında bu jeton güvenli biçimde saklanır ve sonraki tüm API isteklerinin yetkilendirme başlığına (Authorization header) eklenir. Bu yapı, sunucunun her

istik için oturum durumu tutmasını gerektirmeyen, durumsuz (stateless) ve ölçeklenebilir bir kimlik doğrulama sağlar.

Başarısız giriş denemelerinde kullanıcıya, hangi alanın hatalı olduğunu açık etmeyen genel bir hata mesajı gösterilir. Bu, kötü niyetli kişilerin sistemde kayıtlı e-postaları tespit etmesini zorlaştıran bilinçli bir güvenlik önlemidir.

5.2. Kontrol Paneli (Dashboard) Modülü

Başarılı bir girişin ardından kullanıcı, sistemin kalbi olan kontrol paneline yönlendirilir. Kontrol paneli, işletmenin anlık durumunu tek bakışta özetleyen bir komuta merkezi işlevi görür. Dağınık verileri anlamlı göstergelere (KPI) dönüştürerek yöneticinin hızlı ve bilinçli kararlar almasını destekler. Şekil 5.2, kontrol panelini aydınlık temada göstermektedir.



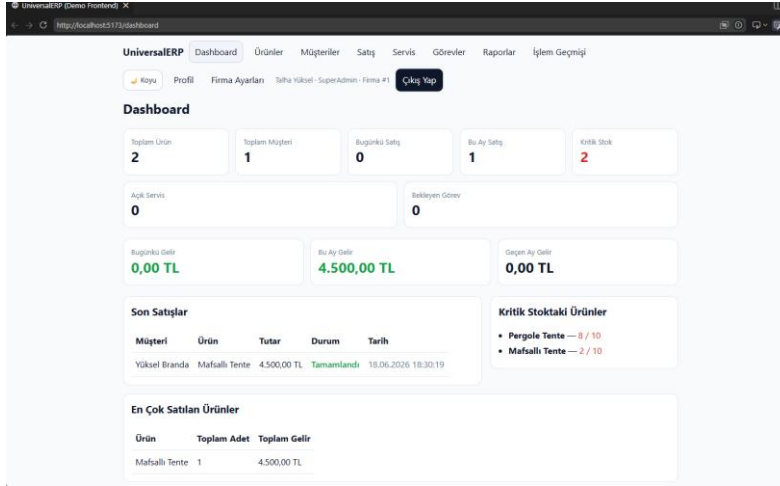
Şekil 5.2. Kontrol paneli - aydınlık tema

Panelin üst kısmında, işletmenin temel metriklerini gösteren özet kartları yer alır: toplam ürün sayısı, toplam müşteri sayısı, günlük ve aylık satış adedi ile kritik stok seviyesindeki ürün sayısı. Hemen altında, bugünkü, bu ayki ve geçen ayki gelir tutarları finansal bir özet sunar. Bu metrikler, arka planda birden çok tabloyu birleştiren toplama (aggregation) sorgularıyla hesaplanır ve performans için ön belleğe alınır.

Panelin alt bölümünde iki önemli liste bulunur. Son Satışlar listesi, en güncel satış hareketlerini müşteri, ürün, tutar ve durum bilgisiyle gösterir. Kritik Stoktaki Ürünler listesi ise stok seviyesi belirlenen eşik altına düşen ürünleri uyarı amacıyla öne çıkarır. Bu sayede yönetici, stok tükenmeden önce tedarik kararı alabilir. En altta yer alan En Çok Satılan Ürünler tablosu ise ürün bazında satış performansını analiz etme imkânı sunar.

UniversalERP, kullanıcı deneyimini zenginleştirmek amacıyla aydınlık ve karanlık olmak üzere iki tema sunar. Kullanıcının tema tercihi tarayıcıda kalıcı olarak saklanır ve sonraki

girişlerde otomatik olarak uygulanır. Şekil 5.3, aynı kontrol panelini karanlık temada göstermektedir. Karanlık tema, özellikle düşük ışıklı ortamlarda göz yorgunluğunu azaltır.



Şekil 5.3. Kontrol paneli - karanlık tema

Ekranın üst kısmındaki gezinme çubuğunda, kullanıcının adı, rolü (örnekte SuperAdmin) ve ait olduğu firma bilgisi gösterilir. Bu bilgi, çok kiracılı yapıda kullanıcının hangi bağlamda işlem yaptığını her an görebilmesini sağlar. Gezinme çubuğundaki menü öğeleri, kullanıcının rolüne göre dinamik olarak biçimlenir; yetkisi olmayan modüller menüde gösterilmez.

5.3. Ürün Yönetimi Modülü

Ürün yönetimi modülü, işletmenin sattığı ürün veya hizmetlerin kataloğunu yönettiği bölümdür. Bu modül, ürünlerin eklenmesi, düzenlenmesi, silinmesi ve listelenmesi gibi temel CRUD (Create, Read, Update, Delete) işlemlerini kapsar. Şekil 5.4, ürün listeleme ekranını göstermektedir.

Şekil 5.4. Ürün yönetimi ekranı ve kritik stok uyarıları

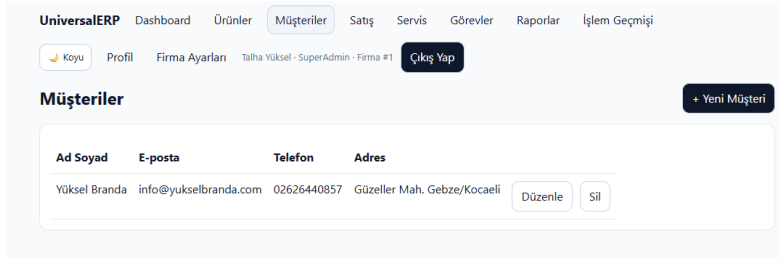
Ekranda her ürün için ad, fiyat, stok adedi ve durum bilgisi bir tablo halinde sunulur. Modülün dikkat çeken bir özelliği, stok yönetimini görsel olarak desteklemesidir. Stok seviyesi kritik eşiğin altına düşen ürünler, yanlarında bir uyarı simgesiyle ve kırmızı renkle vurgulanır. Örnekte görülen iki ürünün de stok değerleri kritik seviyede olduğundan uyarı işaretiyle gösterilmiştir. Bu görsel geri bildirim, yöneticinin stok durumunu hızlıca değerlendirmesini sağlar.

Sağ üst köşedeki Yeni Ürün düğmesi, yeni bir ürün ekleme formunu açar. Her ürün satırındaki Düzenle ve Sil düğmeleri ise ilgili ürün üzerinde işlem yapmayı sağlar. Silme işlemi, daha önce belirtildiği gibi, kaydı fiziksel olarak silmek yerine pasife alan yumuşak silme yaklaşımıyla gerçekleştirilir. Tüm bu işlemler, arka planda ilgili API uç noktalarına yapılan isteklerle yürütülür ve her değişiklik denetim günlüğüne kaydedilir.

Ürün ekleme ve düzenleme işlemlerinde, girilen verilerin geçerliliği hem istemci hem de sunucu tarafında doğrulanır. Örneğin fiyatın negatif olamaması, ürün adının boş bırakılamaması gibi kurallar FluentValidation ile sunucu tarafında merkezi olarak tanımlanmıştır. Bu çift katmanlı doğrulama, hem hızlı kullanıcı geri bildirimi hem de güvenilir veri bütünlüğü sağlar.

5.4. Müşteri Yönetimi Modülü

Müşteri yönetimi modülü, işletmenin ticari ilişki içinde olduğu müşterilerin bilgilerini merkezi bir biçimde tutar. Bu modül, satış işlemlerinin temelini oluşturur; çünkü her satış bir müşteriyle ilişkilendirilir. Şekil 5.5, müşteri listeleme ekranını göstermektedir.



Şekil 5.5. Müşteri yönetimi ekranı

Ekranda her müşteri için ad-soyad, e-posta, telefon ve adres bilgileri listelenir. Sağ üstteki Yeni Müşteri düğmesiyle yeni kayıt eklenebilir; her satırdaki Düzenle ve Sil düğmeleriyle mevcut kayıtlar yönetilebilir. Çok kiracılı yapının bir gereği olarak, bu listede yalnızca giriş yapmış kullanıcının ait olduğu firmaya kayıtlı müşteriler görüntülenir. Global Query Filters mekanizması sayesinde, başka bir firmanın müşterilerinin bu listede görünmesi altyapısal olarak imkânsızdır.

Müşteri kayıtları, satış modülüyle doğrudan ilişkilidir. Yeni bir satış oluşturulurken, müşteri bu modülde tanımlı kayıtlar arasından seçilir. Bu ilişkisel yapı, bir müşterinin satış geçmişinin izlenebilmesini ve müşteri bazlı analizlerin yapılabilmesini mümkün kılar. Müşteri silme işleminde, eğer müşterinin ilişkili satış kayıtları varsa, referans bütünlüğünü korumak adına yumuşak silme uygulanır.

5.5. Satış Yönetimi Modülü

Satış yönetimi modülü, sistemin gelir üreten çekirdek iş sürecini yönetir. Bu modül, müşteri ve ürün modüllerini bir araya getirerek somut satış işlemlerine dönüştürür. Şekil 5.6, satış listeleme ekranını göstermektedir.

Müşteri	Ürün	Adet	Tutar	Ödeme	Durum	Tarih
Yüksel Branda	Mafsallı Tente	1	4.500,00 TL	Havale	Tamamlandı	18.06.2026 18:30:19

Şekil 5.6. Satış yönetimi ekranı

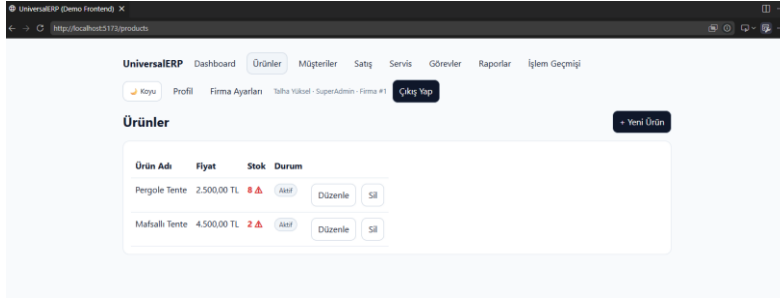
Satış listesinde her kayıt için müşteri, ürün, adet, tutar, ödeme yöntemi, durum ve tarih bilgileri gösterilir. Örnekte görülen satış, Yüksel Branda müşterisine yapılan, Havale ödeme yöntemiyle gerçekleştirilmiş ve durumu Tamamlandı olarak işaretlenmiş bir Mafsallı Tente satışıdır. Satışın durumu renkli bir etiketle vurgulanarak hızlı ayırt edilebilirlik sağlanır.

Yeni bir satış oluşturma süreci, çok kiracılı mimarinin ve iş mantığı katmanının birlikte çalıştığı iyi bir örnektir. Kullanıcı Yeni Satış düğmesine bastığında, müşteri ve ürün seçimi yapacağı bir form açılır. Satış kaydedildiğinde, SaleService içindeki iş mantığı devreye girer: önce seçilen ürünün stok yeterliliği denetlenir, ardından satış kaydı oluşturulur, ürün stoğu düşülür ve bu işlemlerin tamamı Unit of Work deseni sayesinde tek bir veritabanı işleminde atomik olarak kaydedilir. Eş zamanlı olarak işlem, denetim günlüğüne de yazılır.

Her satış satırındaki İptal Et düğmesi, satışın iptal edilmesini sağlar. İptal işlemi, satışın durumunu güncellemenin yanı sıra, düşülen stoğun geri eklenmesi gibi telafi edici iş mantığını da tetikleyebilir. Bu, satış sürecinin yalnızca bir veri kaydından ibaret olmadığını, arka planda tutarlılığı koruyan bir iş akışı olduğunu gösterir.

5.6. Firma Ayarları ve Kiracı Yönetimi

Firma ayarları modülü, çok kiracılı mimarinin kullanıcıya yansıyan yüzüdür. Bu modülde, giriş yapmış kullanıcının ait olduğu firmanın (kiracının) bilgileri görüntülenir ve düzenlenir. Şekil 5.7, firma ayarları ekranını göstermektedir.



Şekil 5.7. Firma ayarları ve kiracı bilgileri ekranı

Ekranda firmanın adı, vergi numarası, faaliyet gösterdiği sektör ve adres bilgileri düzenlenebilir alanlar olarak sunulur. Ayrıca firmanın mevcut abonelik planı (örnekte SaaS Admin) da görüntülenir. Bu bilgiler, her firma için ayrı tutulan kiracı kaydının (Tenant) bir yansımasıdır. Bir kullanıcının burada yaptığı değişiklikler yalnızca kendi firmasını etkiler; başka firmaların ayarlarına erişimi yoktur.

Bu modül, çok kiracılı yapının yönetsel boyutunu temsil eder. Hizmet sağlayıcı (Super Admin) açısından bakıldığında, her yeni firma sisteme bir kiracı olarak eklenir ve kendi yöneticisiyle (Admin) ilişkilendirilir. Böylece tek bir uygulama örneği üzerinden, her biri kendi verisini ve ayarlarını yöneten çok sayıda bağımsız firma barındırılabilir. Bu, SaaS (Hizmet Olarak Yazılım) iş modelinin teknik temelini oluşturur.

5.7. İşlem Geçmişi (Audit Log) Modülü

Kurumsal yazılımlarda hesap verebilirlik ve izlenebilirlik vazgeçilmez gereksinimlerdir. Bir veri kaydının ne zaman, kim tarafından ve nasıl değiştirildiğinin bilinmesi; hem güvenlik denetimleri hem de olası hataların geriye dönük incelenmesi açısından kritik öneme sahiptir. UniversalERP, bu ihtiyacı kapsamlı bir denetim günlüğü (audit log) mekanizmasıyla karşılar. Şekil 5.8, işlem geçmişi ekranını göstermektedir.

UniversalERP Dashboard Ürünler Müşteriler Satış Servis Görevler Raporlar İşlem Geçmişi

Konu Profil Firma Ayarları Talha Yüksel - SuperAdmin - Firma #1 Çıkış Yap

İşlem Geçmişi (Audit Log)

Varlık Adı: Örn: Sale, Product İşlem: Tümü

Filtrele

Kullanıcı	İşlem	Varlık	Kayıt ID	Tarih
-	Oluşturma	Sale	1	18.06.2026 15:30:19

Toplam 1 kayıt - Sayfa 1/1

Önceki Sonraki

Şekil 5.8. İşlem geçmişi (audit log) ekranı

İşlem geçmişi ekranı, sistemde gerçekleşen kritik veri değişikliklerini kullanıcı, işlem türü, etkilenen varlık, kayıt kimliği ve tarih bilgisiyle birlikte listeler. Örnekte, bir Sale (satış) varlığı üzerinde gerçekleştirilen bir Oluşturma işlemi tarih damgasıyla birlikte görülmektedir. Ekranın üst kısmında yer alan filtreleme alanları, kullanıcının kayıtları varlık adına veya işlem türüne göre süzmesine olanak tanır; bu da binlerce kayıt arasında aranan bilgiye hızla ulaşmayı sağlar.

Denetim günlüğü kayıtları, uygulamanın iş mantığı katmanında, ilgili oluşturma, güncelleme ve silme işlemleriyle eş zamanlı olarak otomatik biçimde üretilir. Her kayıt; işlemi yapan kullanıcıyı, işlem tarihini, etkilenen varlığın adını ve kimliğini, işlemin türünü ve mümkün olduğunda değişiklik öncesi ile sonrası değerleri içerir. Bu kapsamlı kayıt yapısı, sistemin tüm geçmişinin geriye dönük olarak yeniden izlenebilmesine olanak tanır.

Bu modül, aynı zamanda KVKK ve GDPR gibi veri koruma düzenlemelerine uyum açısından da değerlidir; çünkü kişisel veriler üzerinde gerçekleştirilen işlemlerin kayıt altına alınmasını ve gerektiğinde raporlanmasını mümkün kılar. Sayfalama (pagination) desteği sayesinde, denetim kayıtlarının hacmi büyüye dahi ekran performansı korunur.

5.8. Servis ve Görev Yönetimi Modülleri

Ürün satışının yanı sıra, birçok işletme satış sonrası hizmet (servis) ve dahili görev takibi gibi süreçleri de yönetmek durumundadır. UniversalERP, bu ihtiyaçları karşılamak üzere servis siparişleri ve görev yönetimi modüllerini de içermektedir. Servis modülü, müşterilere sunulan teknik destek, bakım veya kurulum gibi hizmetlerin kaydedilmesini ve durumlarının (beklemede, devam ediyor, tamamlandı) takip edilmesini sağlar.

Görev yönetimi modülü ise işletme içi iş akışlarının düzenlenmesine yardımcı olur. Çalışanlara atanan görevler, öncelik ve son tarih bilgileriyle birlikte kaydedilir; tamamlanma durumları izlenir. Kontrol panelinde gösterilen Bekleyen Görev metriği, bu modülden beslenerek yöneticiye dikkat edilmesi gereken iş yükü hakkında anlık bilgi sunar. Bu iki modül de, diğer modüller gibi, çok kiracılı izolasyon ve denetim günlüğü mekanizmalarından eksiksiz biçimde yararlanır.

5.9. REST API Uç Noktalarına Genel Bakış

Önceki alt bölümlerde modüllerin işlevleri ele alınmıştır. Bu işlevlerin tamamı, arka planda iyi tanımlanmış REST API uç noktaları aracılığıyla sunulmaktadır. Sistem, toplamda altmıştan fazla uç nokta içermektedir. Çizelge 5.1, her modüle ait temsili uç noktalardan bir bölümünü, kullandıkları HTTP metoduyla birlikte özetlemektedir.

Çizelge 5.1. REST API uç noktalarından örnekler

HTTP Metodu	Uç Nokta (Endpoint)	İşlev
POST	/api/auth/login	Kullanıcı girişi ve jeton üretimi
GET	/api/dashboard	Kontrol paneli özet metrikleri
GET	/api/products	Ürünleri sayfalı biçimde listele
POST	/api/products	Yeni ürün oluştur
PUT	/api/products/{id}	Mevcut ürünü güncelle
DELETE	/api/products/{id}	Ürünü pasife al (soft delete)
GET	/api/customers	Müşterileri listele
POST	/api/sales	Yeni satış oluştur
GET	/api/auditlogs	Denetim kayıtlarını filtreli getir
GET	/api/tenants	Kiracı (firma) bilgilerini getir

Bu uç noktaların her biri, ilgili yetkilendirme öznitelikleriyle korunmaktadır. Örneğin, denetim kayıtlarına erişim yalnızca yönetici rollerine açıkken, satış oluşturma işlemi satış personeline de izin verir. API'nin tamamı, geliştirme ortamında Swagger arayüzü üzerinden

interaktif olarak görüntülenebilir ve test edilebilir; bu da hem geliştirme hem de entegrasyon süreçlerini önemli ölçüde kolaylaştırır.

EK BÖLÜM: DAĞITIM VE ÇALIŞTIRMA ORTAMI

Geliştirilen bir yazılımın değeri, ancak çalışır biçimde dağıtıldığında ortaya çıkar. Bu bölümde, UniversalERP'nin geliştirme ve üretim ortamlarında nasıl yapılandırıldığı ve çalıştırıldığı açıklanmaktadır.

Geliştirme Ortamı

Geliştirme sürecinde, sunucu ve istemci uygulamaları yerel makinede ayrı süreçler olarak çalıştırılmıştır. Sunucu tarafı (ASP.NET Core API), .NET geliştirme sunucusu üzerinde; istemci tarafı (Vue.js) ise Vite geliştirme sunucusu üzerinde koşturulmuştur. Vite'in sıcak modül değiştirme (Hot Module Replacement) özelliği sayesinde, arayüzde yapılan değişiklikler sayfanın yeniden yüklenmesine gerek kalmadan anında görüntülenebilmiş; bu da geliştirme verimliliğini önemli ölçüde artırmıştır. Ekran görüntülerinde görülen localhost:5173 adresi, bu yerel geliştirme sunucusuna aittir.

Yapılandırma Yönetimi

Uygulamanın farklı ortamlarda (geliştirme, test, üretim) farklı ayarlarla çalışması gerekir. Örneğin, geliştirme ortamındaki veritabanı bağlantısı ile üretim ortamındaki bağlantı farklıdır. UniversalERP, bu ihtiyacı ortam tabanlı yapılandırma dosyalarıyla yönetir. Hassas olmayan ayarlar yapılandırma dosyalarında tutulurken; veritabanı parolaları ve JWT imzalama anahtarı gibi hassas bilgiler, üretim ortamında ortam değişkenleri veya güvenli bir sır yönetim hizmeti aracılığıyla sağlanır. Böylece hassas bilgilerin kaynak koduna sızması önlenir.

Bulut Tabanlı Dağıtım Yaklaşımı

Sistemin üretim ortamı için bulut tabanlı bir dağıtım mimarisi öngörülmüştür. Bu mimaride, API uygulaması bir uygulama hizmeti (App Service) üzerinde barındırılır ve gelen trafik bir yük dengeleyici (load balancer) aracılığıyla birden çok uygulama örneğine dağıtılır. Bu yapı, hem yüksek erişilebilirlik (bir örnek arızalansa dahi hizmetin devam etmesi) hem de yatay ölçeklenebilirlik (yük arttığında örnek sayısının artırılması) sağlar.

Veritabanı katmanı, yüksek erişilebilirlik yapılandırmasıyla çalışan yönetilen bir SQL Server hizmeti üzerinde konumlandırılır; otomatik yedekleme ve coğrafi yedeklilik özellikleriyle veri kaybı riski en aza indirilir. Durumsuz JWT tabanlı kimlik doğrulamanın bu mimarideki kritik rolü bir kez daha öne çıkar: oturum durumu sunucuda tutulmadığından,

bir kullanıcının ardışık istekleri farklı uygulama örneklerine düşse dahi sistem tutarlı biçimde çalışır. Bu da yük dengelemenin sorunsuz işlemlerini mümkün kılar.

6. GÜVENLİK VE YETKİLENDİRME

Kurumsal verilerin işlendiği bir ERP sisteminde güvenlik, sonradan eklenen bir özellik değil; tasarımın her aşamasına nüfuz etmesi gereken temel bir gereksinimdir. UniversalERP, derinlemesine savunma (defense in depth) ilkesi doğrultusunda birden çok güvenlik katmanı içerir. Bu bölümde, sistemin kimlik doğrulama, yetkilendirme ve veri güvenliği mekanizmaları ele alınmaktadır.

6.1. JWT Tabanlı Kimlik Doğrulama

Sistemin kimlik doğrulama altyapısı, JSON Web Token (JWT) standardı üzerine kuruludur. JWT, taraflar arasında güvenli biçimde bilgi taşımaya sağlayan, dijital olarak imzalanmış ve kendi kendine yeten (self-contained) bir jeton biçimidir. Bir JWT üç bölümden oluşur: jetonun türünü belirten başlık (header), kullanıcı bilgilerini taşıyan yük (payload) ve jetonun bütünlüğünü güvence altına alan imza (signature).

Kullanıcı başarıyla giriş yaptığında, sunucu yük bölümüne kullanıcının kimliğini, kiracı bilgisini ve rollerini yerleştirir; ardından bu içeriği gizli bir anahtarla imzalar. İmza sayesinde, jetonun içeriği değiştirildiğinde bu durum sunucu tarafından anında tespit edilir. Jeton, belirli bir süre sonunda geçerliliğini yitirir; bu da çalınan bir jetonun süresiz kullanılmasını engeller.

Bu yaklaşımın en önemli avantajı durumsuz (stateless) olmasıdır. Sunucu, her kullanıcı için oturum bilgisi saklamak zorunda kalmaz; çünkü gerekli tüm bilgi jetonun kendisinde taşınır. Bu özellik, sistemin yatay olarak ölçeklenmesini, yani yük arttığında birden çok sunucu örneğiyle hizmet verilmesini büyük ölçüde kolaylaştırır.

6.2. Rol Tabanlı Erişim Denetimi (RBAC)

Kimlik doğrulama kullanıcının kim olduğunu belirlerken, yetkilendirme o kullanıcının neyi yapmaya izinli olduğunu belirler. UniversalERP, yetkilendirme için rol tabanlı erişim denetimi (RBAC) modelini kullanır. Üçüncü bölümde tanımlanan altı rol (Super Admin, Admin, Manager, Staff, Viewer, Guest), her birinin yapabileceği işlemleri net biçimde sınırlar.

Yetkilendirme, sistemde iki düzeyde uygulanır. Birinci düzeyde, API denetleyicilerine veya tekil uç noktalara eklenen yetkilendirme öznitelikleri (örneğin [Authorize(Roles = "Admin,Manager")]) sayesinde, yalnızca belirtilen rollere sahip kullanıcılar ilgili uç noktaya erişebilir. İkinci düzeyde ise iş mantığı katmanında daha ayrıntılı (granular) denetimler

yapılarak, rolün ötesinde bağlama özgü kurallar (örneğin bir kullanıcının yalnızca kendi kiracısının verisine erişebilmesi) uygulanır.

Yetkilendirme yalnızca sunucu tarafında değil, kullanıcı arayüzünde de yansıtılır. Kullanıcının rolüne göre, erişim yetkisi olmayan menü öğeleri ve düğmeler arayüzde hiç gösterilmez veya pasif hâle getirilir. Ancak arayüzdeki bu kısıtlamaların yalnızca kullanıcı deneyimini iyileştirmeye yönelik olduğu, asıl güvenlik denetiminin her zaman sunucu tarafında yapıldığı önemle vurgulanmalıdır. Çünkü istemci tarafı denetimler kötü niyetli kullanıcılar tarafından atlatılabilir.

6.3. Veri Güvenliği Önlemleri

Sistem, yaygın web güvenlik açıklarına karşı çok yönlü önlemler içerir:

- **Parola Güvenliği:** Kullanıcı parolaları asla düz metin olarak saklanmaz. Her parola, sektör standardı bir algoritma olan BCrypt ile, geri döndürülemez biçimde özetlenerek saklanır. BCrypt'in uyarlanabilir maliyet faktörü, donanım güçlendikçe kaba kuvvet saldırılarına karşı direncin artırılmasına olanak tanır.
- **SQL Enjeksiyonu Koruması:** Tüm veritabanı işlemleri Entity Framework Core üzerinden, parametrelili sorgularla gerçekleştirilir. Bu yapı, kullanıcı girdisinin doğrudan SQL sorgusuna gömülmesini engelleyerek SQL enjeksiyonu saldırılarını yapısal olarak önler.
- **XSS (Cross-Site Scripting) Koruması:** Vue.js, arayüzde gösterilen tüm dinamik içeriği varsayılan olarak güvenli biçimde kaçışlar (escape). Bu sayede kullanıcı verisi içine gizlenmiş kötü amaçlı betiklerin çalıştırılması engellenir.
- **Hassas Verilerin Korunması:** Veritabanı bağlantı dizeleri ve JWT imzalama anahtarı gibi hassas yapılandırma bilgileri, kaynak koda gömülme yerine, üretim ortamında güvenli bir sır yönetim hizmetinde (örneğin Azure Key Vault) saklanır.
- **Kiracı İzolasyonu:** Çok kiracılı yapıda Global Query Filters mekanizması, bir kiracının verisine başka bir kiracının erişmesini altyapısal olarak imkânsız kılarak en kritik veri güvenliği garantilerinden birini sağlar.

6.4. Tehdit Modeli ve Savunma Katmanları

Güvenliği sistematik biçimde ele almak için, sistemin karşı karşıya olabileceği temel tehditlerin tanımlanması ve her birine karşı uygun savunma katmanlarının konumlandırılması gerekir. UniversalERP'nin güvenlik tasarımı, yaygın web uygulaması tehditlerini hedef alan çok katmanlı bir savunma yaklaşımı benimser. Aşağıdaki çizelge, başlıca tehdit türlerini ve bunlara karşı alınan önlemleri özetlemektedir.

Çizelge 6.1. Tehdit türleri ve uygulanan savunma önlemleri

Tehdit Türü	Uygulanan Savunma Önlemi
Yetkisiz Erişim	JWT kimlik doğrulama ve rol tabanlı erişim denetimi
Parola Sızıntısı	BCrypt ile geri döndürülemez parola özetleme
SQL Enjeksiyonu	EF Core parametrelili sorgular
XSS Saldırıları	Vue.js otomatik içerik kaçıışı
Kiracılar Arası Sızıntı	Global Query Filters ile otomatik izolasyon
Jeton Hırsızlığı	Kısa ömürlü jetonlar ve imza doğrulaması
Hassas Veri İfşası	Sır yönetimi ve standart hata yanıtları

Bu katmanlı yaklaşımın temel felsefesi, hiçbir tek savunma mekanizmasına mutlak güven duymamaktır. Derinlemesine savunma (defense in depth) ilkesi gereği, bir katman aşılsa dahi diğer katmanların devreye girerek sistemi koruması hedeflenir. Örneğin, istemci tarafı arayüz kısıtlamaları atlatılsa bile, sunucu tarafındaki yetkilendirme denetimi yetkisiz işlemi engeller; bir sorguda kiracı filtresi unutulsa dahi, Global Query Filters bu boşluğu kapatır. Bu çok katmanlı yapı, sistemin güvenlik dayanıklılığını önemli ölçüde artırır.

7. TEST VE PERFORMANS DEĞERLENDİRMESİ

Bir yazılımın doğru çalıştığından emin olmanın tek yolu, onu sistematik biçimde sınamaktır. Benzer şekilde, sistemin gerçek dünya yükü altında kabul edilebilir bir performans sergilediğini göstermek de üretim ortamına geçiş için kritik öneme sahiptir. Bu bölümde, UniversalERP'nin test stratejisi ve performans değerlendirmesi ele alınmaktadır.

7.1. Test Stratejisi

Sistemin doğruluğunu güvence altına almak için, yazılım test piramidi yaklaşımı benimsenmiştir. Bu yaklaşımda, hızlı ve sayıca çok olan birim testleri tabanı oluştururken; daha kapsamlı ancak yavaş olan entegrasyon ve uçtan uca testler piramidin üst katmanlarını oluşturur.

7.1.1. Birim Testleri (Unit Tests)

Birim testleri, sistemin en küçük mantıksal parçalarını (genellikle servis metodlarını) izole biçimde sınar. Bu testler, xUnit test çatısı kullanılarak yazılmıştır. Test edilen servisin bağımlılıkları (örneğin depolar), Moq kütüphanesi ile taklit edilerek (mock) gerçek veritabanına ihtiyaç duyulmadan, yalnızca iş mantığının doğruluğu sınanmıştır. Örneğin, bir satış oluşturulurken stok yetersizse işlemin reddedilmesi gerektiği, birim testleriyle doğrulanmıştır. Doğrulama ifadeleri, okunabilirliği artırmak için FluentAssertions kütüphanesiyle yazılmıştır.

7.1.2. Entegrasyon Testleri (Integration Tests)

Entegrasyon testleri, birden çok bileşenin birlikte doğru çalışıp çalışmadığını sınar. Bu testlerde, gerçek veritabanı yerine bellek içi (in-memory) bir veritabanı sağlayıcısı kullanılarak, veri erişim katmanının ve iş mantığının veritabanıyla birlikte beklendiği gibi davrandığı doğrulanmıştır. Böylece, bir müşteri eklendiğinde kaydın gerçekten oluşturulduğu ve geri okunabildiği uçtan uca teyit edilmiştir.

7.1.3. Test Kapsamı

Geliştirme sürecinde, kritik iş mantığını içeren servis ve doğrulayıcı (validator) sınıfları öncelikli olarak test edilmiştir. Test kapsamı (code coverage), kodun ne kadarının testlerle çalıştırıldığını gösteren bir ölçüttür ve sistemin çekirdek iş mantığı için yüksek bir kapsama oranı hedeflenmiştir. Testler, sürüm kontrol sistemine yapılan her gönderimde otomatik olarak çalıştırılarak, yeni eklenen değişikliklerin mevcut işlevselliği bozmadığı (regresyon) sürekli güvence altına alınmıştır.

7.2. Performans Analizi

Sistemin yük altındaki davranışını değerlendirmek için, eş zamanlı çok sayıda kullanıcının sistemi kullandığı senaryolar simüle edilmiştir. Performans testlerinde, API uç noktalarına artan sayıda eş zamanlı istek gönderilerek sistemin yanıt süresi, verimi (throughput) ve hata oranı ölçülmüştür. Çizelge 7.1, temsili yük testi sonuçlarını özetlemektedir.

Çizelge 7.1. Yük testi performans sonuçları

Metrik	Değer	Açıklama
Ortalama Yanıt Süresi	~250 ms	Tipik istek yanıt süresi
95. Yüzdelik Dilim	~450 ms	İsteklerin %95'i bu sürenin altında
Hata Oranı	< %1	Yük altında başarısız istek oranı
Eş Zamanlı Kullanıcı	1000	Test edilen eş zamanlı kullanıcı sayısı

Elde edilen sonuçlar, sistemin hedeflenen kullanıcı kitlesi olan KOBİ'lerin iş yüklerini rahatlıkla karşılayabileceğini göstermektedir. Performansın bu düzeyde tutulmasında, dördüncü bölümde anlatılan veritabanı optimizasyonlarının (uygun indeksleme, N+1 probleminin çözümü, önbellekleme) belirleyici rol oynadığı gözlemlenmiştir. Ayrıca durumsuz JWT tabanlı kimlik doğrulama, sistemin gerektiğinde birden çok sunucu örneğiyle yatay olarak ölçeklenmesine olanak tanıyarak, gelecekteki yük artışlarına karşı bir esneklik sağlamaktadır.

7.3. Kod Kalitesi ve Sürdürülebilirlik Değerlendirmesi

Bir yazılımın kalitesi yalnızca çalışıp çalışmadığıyla değil; ne kadar kolay anlaşıldığı, değiştirildiği ve genişletildiğiyle de ölçülür. UniversalERP'nin geliştirilmesinde, kod kalitesini yüksek tutmak için çeşitli pratikler uygulanmıştır. Bunların başında, üçüncü bölümde detaylandırılan Clean Architecture ve SOLID prensiplerine bağlılık gelir. Bu prensipler sayesinde, sistemin herhangi bir parçasında yapılacak değişikliğin diğer parçalara olan etkisi en aza indirilmiştir.

Kod genelinde tutarlı bir adlandırma kuralı benimsenmiş; sınıf, metot ve değişken isimleri, işlevlerini açıkça ifade edecek biçimde seçilmiştir. Bu, kodun kendi kendini belgeleyen (self-documenting) bir nitelik kazanmasını sağlamıştır. Tekrar eden mantık, ortak yardımcı sınıflar ve temel sınıflar (örneğin BaseEntity) altında toplanarak, Kendini Tekrar Etme (Don't Repeat Yourself) ilkesine uyulmuştur.

Sürdürülebilirliğin bir diğer önemli göstergesi, sisteme yeni bir özelliğin ne kadar kolay eklenebildiğidir. UniversalERP'nin katmanlı ve modüler yapısı sayesinde, yeni bir iş varlığının (örneğin bir tedarikçi modülü) sisteme eklenmesi, mevcut kodu değiştirmeden, yerleşik desenlerin (varlık, depo, servis, denetleyici) tekrar uygulanmasıyla mümkündür. Bu genişletilebilirlik, sistemin gelecekteki gereksinimlere uyum sağlama kapasitesinin somut bir kanıtıdır.

7.3.1. Statik Kod Analizi

Geliştirme sürecinde, derleyici uyarıları ve statik kod analizi araçları etkin biçimde kullanılmıştır. Bu araçlar, çalışma zamanında ortaya çıkabilecek olası hataları (örneğin boş referans erişimleri) daha kod yazılırken tespit ederek erken müdahaleyi mümkün kılar. Potansiyel sorunların geliştirme aşamasında giderilmesi, bunların üretim ortamına taşınma riskini önemli ölçüde azaltır ve genel kod sağlığını korur.

Sonuç olarak, test stratejisi ve kod kalitesi pratiklerinin bir bütün olarak uygulanması, UniversalERP'yi yalnızca işlevsel değil; aynı zamanda güvenilir, bakımı yapılabilir ve uzun vadede sürdürülebilir bir sistem haline getirmiştir. Bu nitelikler, gerçek bir üretim ortamında çalışması beklenen kurumsal bir yazılım için vazgeçilmez gereksinimlerdir.

8. SONUÇLAR VE ÖNERİLER

Bu bitirme çalışması kapsamında, çok kiracılı mimariye sahip, bulut tabanlı ve tam işlevsel bir kurumsal kaynak planlama sistemi olan UniversalERP, tasarım aşamasından gerçekleştirime kadar uçtan uca geliştirilmiştir. Bu son bölümde, çalışmadan elde edilen sonuçlar değerlendirilmekte, geliştirme sürecinde karşılaşılan zorluklar tartışılmakta ve gelecekte yapılabilecek çalışmalara yönelik öneriler sunulmaktadır.

8.1. Elde Edilen Sonuçlar

Çalışmanın başında belirlenen hedeflerin tamamına ulaşılmıştır. Geliştirilen sistemle aşağıdaki somut sonuçlar elde edilmiştir:

- Clean Architecture prensiplerine dayalı, katmanları net biçimde ayrılmış, sürdürülebilir ve test edilebilir bir yazılım mimarisi başarıyla kurulmuştur. Bu mimari, sistemin gelecekte kolayca genişletilebilmesinin önünü açmıştır.
- Tek bir uygulama ve veritabanı örneği üzerinden birden çok firmaya güvenli biçimde hizmet veren çok kiracılı altyapı, Global Query Filters mekanizmasıyla, veri sızıntısı riskini altyapısal olarak ortadan kaldıracak şekilde gerçekleştirilmiştir.
- JWT tabanlı kimlik doğrulama ve altı düzeyli rol tabanlı erişim denetimiyle, kurumsal düzeyde bir güvenlik katmanı oluşturulmuştur.
- Ürün, müşteri, satış, stok takibi, firma ayarları ve denetim günlüğü modülleri eksiksiz biçimde hayata geçirilmiş; tüm bu modüller çalışan bir uygulamada gösterilmiştir.
- Aydınlık ve karanlık tema desteği sunan, duyarlı ve modern bir kullanıcı arayüzü geliştirilerek olumlu bir kullanıcı deneyimi sağlanmıştır.

Bu sonuçlar bir bütün olarak değerlendirildiğinde, modern ve açık kaynaklı teknolojilerin doğru bir mimari yaklaşımla bir araya getirilmesiyle, KOBİ'ler için düşük maliyetli, ölçeklenebilir ve genişletilebilir bir ERP alternatifinin gerçekleştirilebileceği somut biçimde kanıtlanmıştır. Çalışma, akademik bilginin gerçek bir mühendislik ürününe nasıl dönüştürülebileceğine dair bütünsel bir örnek teşkil etmektedir.

8.2. Karşılaşılan Zorluklar

Geliştirme süreci çeşitli teknik zorlukları beraberinde getirmiştir. Çok kiracılı mimarinin kurulması, başlangıçta en çok zaman alan ve dikkat gerektiren konu olmuştur; veri izolasyonunun her senaryoda doğru çalıştığından emin olmak kapsamlı testler

gerektirmiştir. Veri erişiminde karşılaşılan N+1 sorgu problemi, performansı ciddi biçimde etkilemiş ve ilişkili verilerin doğru biçimde birlikte yüklenmesiyle çözülmüştür. İstemci tarafında, modüller arası paylaşılan durumun yönetimi karmaşıktıkça, Pinia depolarının mantıksal olarak iyi tasarlanmış birimlere bölünmesi gerekmiştir. Bu zorlukların her biri, sürecin sonunda değerli birer öğrenme deneyimine dönüşmüştür.

8.3. Gelecek Çalışmalar

UniversalERP, sağlam bir temel sunmakla birlikte, bir ERP sisteminin doğası gereği sürekli geliştirilmeye açıktır. Gelecekte yapılabilecek çalışmalar şu başlıklar altında değerlendirilebilir:

- Finansal Entegrasyonlar: Sanal POS ile gerçek ödeme altyapısı entegrasyonu ve e-fatura/e-arşiv sistemleriyle yasal uyumluluk sağlanabilir.
- Mobil Uygulama: Mevcut API altyapısı korunarak, iOS ve Android platformları için yerel (native) veya çapraz platform bir mobil istemci geliştirilebilir.
- Gelişmiş Analitik ve Yapay Zekâ: Geçmiş satış verileri kullanılarak talep tahminlemesi, stok optimizasyonu ve müşteri segmentasyonu gibi yapay zekâ destekli özellikler eklenebilir.
- Mikroservis Mimarisine Geçiş: Sistem ölçeklendikçe, mevcut modüler yapı, bağımsız olarak ölçeklenebilen ve dağıtılabilen mikroservislere ayrıştırılabilir.
- Gerçek Zamanlı Bildirimler: Halihazırda altyapısı bulunan SignalR teknolojisi genişletilerek, kritik stok düşüşü veya yeni satış gibi olaylarda kullanıcılara anlık bildirimler sunulabilir.

Sonuç olarak bu çalışma, hem teknik bir ürün ortaya koymuş hem de yazılım mühendisliğinin temel prensiplerinin uygulamalı olarak deneyimlenmesine olanak tanıyan kapsamlı bir öğrenme süreci olmuştur. Geliştirilen sistem, sağlam mimari temeli sayesinde, yukarıda sıralanan gelecek çalışmalar için elverişli bir başlangıç noktası niteliğindedir.

KAYNAKLAR

- [1] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [2] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- [3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [4] Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
- [5] Fowler, M. (2012). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- [6] Microsoft. (2024). ASP.NET Core Documentation. Erişim adresi: <https://learn.microsoft.com/aspnet/core>
- [7] Microsoft. (2024). Entity Framework Core Documentation. Erişim adresi: <https://learn.microsoft.com/ef/core>
- [8] Vue.js. (2024). Vue 3 Documentation. Erişim adresi: <https://vuejs.org/guide/>
- [9] Pinia. (2024). Pinia Documentation. Erişim adresi: <https://pinia.vuejs.org/>
- [10] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force.
- [11] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doktora Tezi, University of California, Irvine.
- [12] Chong, F., & Carraro, G. (2006). Architecture Strategies for Catching the Long Tail. Microsoft Corporation.
- [13] Newman, S. (2015). Building Microservices: Designing Fine-Grained Systems. O'Reilly Media.
- [14] Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference.
- [15] Microsoft. (2024). Microsoft SQL Server Documentation. Erişim adresi: <https://learn.microsoft.com/sql/>

ÖZGEÇMİŞ

Kişisel Bilgiler

Ad Soyad: Talha YÜKSEL

Doğum Yeri ve Tarihi: [Pendik, 30.06.2003]

E-posta: [yuksektalha7@gmail.com]

Eğitim Durumu

Lisans (2022 – 2026): Bursa Teknik Üniversitesi, Mühendislik ve Doğa Bilimleri Fakültesi, Bilgisayar Mühendisliği Bölümü

Teknik Beceriler

- Programlama Dilleri: C#, TypeScript, JavaScript, SQL
- Backend Teknolojileri: ASP.NET Core, Entity Framework Core, RESTful API tasarımı
- Frontend Teknolojileri: Vue.js, Pinia, HTML5, CSS3
- Veritabanı: Microsoft SQL Server, ilişkisel veritabanı tasarımı
- Araçlar ve Yöntemler: Git, GitHub, Docker, Swagger, çevik (agile) geliştirme
- Mimari Yaklaşımlar: Clean Architecture, SOLID prensipleri, tasarım desenleri

İlgi Alanları

- Yazılım mimarisi ve tasarım desenleri
- Bulut bilişim ve dağıtık sistemler
- Veritabanı tasarımı ve performans optimizasyonu